Sd & Sd++ Programming Textbook

From Fundamentals to Advanced Topics

Flat syntax · The `sta` frame and `iid` identity · The `ed` terminator

Terminator flags · Pointers · Reback · Metaprogramming

Standard Edition: Sd 2 / Sd++ 2

Exercises, answers, quick-reference tables, and formal grammar

Language author: Lương
Textbook compilation: Claude (based on the author's specification)

Version 2.0 · November 2026

Document Information

Field	Details
Title	*Sd & Sd++ Programming Textbook (From Fundamentals to Advanced Topics)*
Version	2.0, November 2026
Scope	Sd 2 (core language, Part I) and Sd++ 2 (system mode, Part II)
Specification source	Sd/Sd++ code supplied by the author (the `ssta` / `$` / ₫ / € version and the `sta(main)` / `ed` version captured from screenshots)
Paper size	A4, duplex printing, 3.2 cm inner margin and 2.2 cm outer margin
Colors	Black and shades of gray only; fully readable in black-and-white printing
Fonts	Noto Sans and DejaVu Sans Mono; embedded in the PDF

Translation note

Language keywords and syntax symbols are preserved. Variable, function, and structure identifiers in code examples remain in their original form to avoid changing the examples; human-readable comments and displayed strings are translated into English.

Specification status

At the time this textbook was prepared, Sd/Sd++ had no official compiler or interpreter. Every example in the book was checked with a syntax linter written according to the rules in this book: every instruction has an `ed` terminator, every block is closed with the correct block terminator, and braces are balanced. No example has been run on a real compiler, so the output shown in examples is the output prescribed by the specification, not an experimentally measured result.

How to read the [PR] marker

Syntax already present in the author's code is considered official and is not marked. Material added by the compiler of this textbook to make the language consistent and usable is marked **[PR]** (proposal). A [PR] feature becomes official only after the author approves it. The author may remove or rename any such item without invalidating the rest of the book. Appendix D lists all changes from the original code.

Contents

- Foreword
- Suggested 8-week learning path
- **Part I — Sd Fundamentals**
  - Chapter 1 — Getting Started with Sd
  - Chapter 2 — Variables, Data Types, and Operators
  - Chapter 3 — Input, Output, and Formatting
  - Chapter 4 — Branching: The `bolean` Block
  - Chapter 5 — Loops: `for` and `while`
  - Chapter 6 — Arrays and Strings
  - Chapter 7 — Functions and Procedures: `fn` and `void`
  - Chapter 8 — Data Structures: `struct`
  - Chapter 9 — Error Codes and Debugging
- **Part II — Advanced Sd++**
  - Chapter 10 — Sd++ and the `ssta` / `##` Boundary
  - Chapter 11 — Bare Global Variables and the `define` Frame
  - Chapter 12 — Pointers and Heap Memory
  - Chapter 13 — The `ids sb` Selection Block and Reback
  - Chapter 14 — Metaprogramming: `defk` and Custom Terminator Flags
  - Chapter 15 — Standard Library
  - Chapter 16 — The Sd and Sd++ Compiler
  - Chapter 17 — Integrated Projects
- Appendix A — Keywords and Symbols
- Appendix B — Formal Grammar (EBNF)
- Appendix C — Error and Warning Codes
- Appendix D — Original Specification Compared with Sd 2
- Appendix E — Quick Reference
- Appendix F — Exercise Answers
- Appendix G — Glossary

Foreword

Sd is a programming language built around a flat philosophy: every instruction has the same general shape, one `sta` frame, one `iid` identity, and one `ed` terminator. Readers do not have to memorize dozens of different kinds of brackets and semicolons. They need to remember one pattern and understand how that pattern changes for each task: declaration, printing, input, looping, branching, and functions.

Sd++ is an extension for systems programming: the terminator-flag set `$`, `₫`, `€`, `$$`, `#`, `##`; pointers; Heap memory; the self-checking selection block (`Reback`); and `defk` metaprogramming. Every Sd program is a valid Sd++ program when placed inside an `ssta { ... }` frame.

This book does three things:

1. **Establishes a consistent syntax.** The original screenshot-based code contains several self-contradictions. For example, `==` is used both to connect a specification and to compare values, the `for` loop lacks initialization, and a structure contains two fields with the same name `a`. The book preserves the author's intent and keywords while correcting these issues (Appendix D).
2. **Teaches from the beginning to advanced topics.** Part I (Chapters 1–9) moves from the first program to functions, arrays, structures, and debugging. Part II (Chapters 10–17) teaches Sd++ and ends with two complete projects.
3. **Provides a durable printed reference.** Exercises follow each chapter; answers are in Appendix F; Appendix E contains a one-page quick reference; and Appendix B provides the formal grammar, so the book remains useful after the course has been completed.

Suggested learning path (8 weeks)

Week	Chapters	End-of-week goal
1	1, 2	Write and read programs that use variables and arithmetic
2	3, 4	Read input and branch across multiple cases
3	5	Become comfortable with `for`, `while`, `brk`, and `nxt`
4	6, 7	Use arrays and strings; divide programs into functions
5	8, 9	Work with data structures; read and fix error messages independently
6	10, 11, 12	Sd++: terminator flags, global variables, pointers, and Heap
7	13, 14, 15	Reback, `defk`, and the standard library
8	16, 17	Understand the compiler; complete an entire project

How to study

Re-type each example instead of merely reading it. Complete the exercises before looking at the answers. When an error occurs, consult the error codes in Appendix C before asking someone else.

Part I — Sd Fundamentals

Chapter 1 — Getting Started with Sd

1.1 What is Sd?

Sd is a flat programming language. When you look at an Sd program, its instructions are arranged in a straight line, and each instruction has the same general shape. Three components make up that shape:

- **`sta` (Statement Authority), the body:** builds the frame for an instruction; it is where conditions, loops, functions, and scopes are held.
- **`iid` (Instruction Identity), the soul:** indicates that this is an instruction to be executed (printing, input, assignment, looping, and so on).
- **`ed` (end), the terminator:** marks the end of an instruction or block.

A body without a soul is only an empty frame; a soul without a body has nowhere to execute. Therefore, most Sd instructions begin with `iid:` and contain a `sta(...)` frame.

1.2 Your first program

Example 1.1. Hello program

```
imp*ioprt.ed

sta(main){
    iid: sta(pri) => "Hello, Sd!" ed
    iid: ref* 0 ed
}
```

Screen output:

```
Hello, Sd!
```

Read it line by line:

Line	Meaning
`imp*ioprt.ed`	Loads the input/output library `ioprt`. An import line always has the form `imp*NAME.ed` and appears at the beginning of the file.
`sta(main){`	Opens the main program frame. Every program has exactly one `sta(main)`; execution begins here.
`iid: sta(pri) => "..." ed`	A print instruction. `pri` is the instruction type, `=>` connects the instruction type to its content, and `ed` terminates the instruction.
`iid: ref* 0 ed`	Ends the program and returns code 0 (success). The `main` frame must close with a `ref*` instruction.
`}`	Closes the `main` frame.

Why is `ref*` mandatory?

In the original version, the line `ref* 0 ed` was merely a customary way to end a program. Sd 2 turns it into a rule: the program must explicitly state the value with which it terminates. A value of 0 means everything went correctly; any nonzero value is an error code chosen by you (Chapter 9).

1.3 Anatomy of an instruction

Every Sd instruction follows the same pattern:

```
iid: sta(HEADER) => TAG: sta(BODY) ed
```

The `=> TAG: sta(BODY)` part is present only when the instruction needs additional information. A simple instruction such as `iid: ref* 0 ed` does not need it.

Parts of a declaration with an initial value

```
iid: sta(int a) => edt: sta(a = 5) ed
```

The parts are:

- `iid`: this is an instruction.
- `sta(int a)`: the header; it declares variable `a`.
- `=>`: connects the header to the following part.
- `edt`: the tag meaning “edit the value”.
- `sta(a = 5)`: the frame containing the body (`a = 5`).
- `ed`: the instruction terminator.

Two things to remember:

1. `=>` only connects the header to the body. It is **not** a comparison operator; comparisons use `==` (Chapter 2). In the original version, `==` did both jobs, which was confusing, so Sd 2 separates them.
2. An instruction has only one `ed` terminator, at its end.

1.4 Three kinds of components

A program in Sd contains only three kinds of components:

Type	Recognition	Example
Simple instruction	`iid: ...` `ed` on one line	`iid: sta(pri) => "Hi" ed`

Type	Recognition	Example
Block	`iid: sta(type) => tag:`, multiple lines, closed by the matching block name plus ed	Loops, branches, functions
Comment	Appears between two pairs of `\\`	`\\ this is a comment \\`

Blocks will be introduced gradually from Chapter 4 onward. To make them easy to remember, all blocks follow this pattern:

```
iid: sta(TYPE) => TAG1,TAG2:      <- block opening, ends with a colon
    ...block body...
TYPE-CLOSER ed                   <- closes the block
```

1.5 Comments

A comment is text for human readers; the compiler ignores it. Comments are enclosed in two pairs of backslashes (`\\`) and may occupy one line or several lines.

Example 1.2. Single-line and multi-line comments

```
imp*ioprt.ed

\\ A program that prints two lines.
   A block comment may span several lines. \\
sta(main){
    iid: sta(pri) => "Line 1" ed      \\ print line 1 \\
    iid: sta(pri) => "Line 2" ed
    iid: ref* 0 ed
}
```

Comments must be closed

The original version warns that if you open `\\` and forget to close it, the reader will consume all remaining source code as a comment. Sd 2 does not silently swallow the rest of the file: if the end of the file is reached while a comment is still open, the compiler reports error E101 along with the line where the comment was opened, so you can find the missing closer immediately.

1.6 File and formatting conventions

- Sd source files use UTF-8 encoding and the `.sd` extension [PR].
- Put each simple instruction on its own line; indent by four spaces for each frame level.
- Strings and comments may contain accented characters. Examples in the book omit Vietnamese diacritics to make them easy to type on any keyboard.
- Uppercase and lowercase letters are different: `Ten` and `ten` are two different names.

Chapter summary

Every instruction follows the pattern `iid: sta(...) ... ed`. The `=>` symbol connects the header to the body. A block closes with its matching block-name terminator. The `main` frame closes with `ref*`. Comments are enclosed in two pairs of `\\`.

Exercises

1.1 Write a program that prints your name on one line and your class on the next.
1.2 Add a multi-line comment at the beginning of the program above, recording the date and purpose.
1.3 Which of the following lines has an invalid instruction pattern, and why?

```
iid: sta(pri) => "A"     iid sta(pri) => "B" ed       iid: sta(pri) => "C" ed
```

1.4 What is missing from the following program for it to be valid?

```
sta(main){ iid: sta(pri) => "x" ed }
```

Chapter 2 — Variables, Data Types, and Operators

2.1 What is a variable?

A variable is a named memory cell that contains a value of a particular type. Variables in Sd are declared using the `sta(type name)` frame.

Example 2.1. Declaring variables

```
sta(main){
    iid: sta(int a) ed                       \\ declaration; default a = 0 \\
    iid: sta(int b) => edt: sta(b = 7) ed    \\ declaration with an initial value \\
    iid: ref* 0 ed
}
```

There are two forms:

1. `iid: sta(int a) ed`: declaration only.
2. `iid: sta(int b) => edt: sta(b = 7) ed`: declaration and initial value. The `edt` tag indicates that the body changes the variable's value.

Default values [PR]

A declared variable that has not been assigned a value is never in a "garbage" state: `int` defaults to `0`, `float` to `0.0`, `str` to the empty string `""`, and `bool` to `false`. This ensures that the same program always produces the same result.

2.2 Basic data types

Type	Meaning	Example values	Default
`int`	Integer	`42`, `-7`	`0`
`float`	Floating-point number	`3.14`, `-0.5`	`0.0`
`str`	String	`"Sd++"`	`""`
`bool`	True or false	`true`, `false`	`false`

Sd also has arrays (Chapter 6) and structures (Chapter 8); Sd++ additionally has the pointer type `ptr` (Chapter 12).

2.3 Naming rules

- A name consists of unaccented letters, digits, and underscores; it must not begin with a digit.
- Names must not conflict with keywords (see Appendix A).
- Two variables in the same `{ ... }` frame may not have the same name (error E302).
- Names should communicate meaning: `tong_tien` is better than `tt`.

2.4 Operators

Group	Operators	Notes
Arithmetic	`+ - * / %`	`/` on two integers performs integer division; `%` returns the remainder.
Increment/decrement	`i++`, `i--`	Used only as an instruction or in a loop header.

Group	Operators	Notes
Comparison	`== != < <= > >=`	Produces a `bool` result.
Logic [PR]	`and`, `or`, `not`	Combines Boolean values.
Assignment	`=`	Appears only in an instruction body.

Operator precedence, from highest to lowest (operators at the same level are evaluated from left to right):

Level	Operators
1	`( ) [ ] .`
2	`not`, unary `-`
3	`* / %`
4	`+ -`
5	`< <= > >=`
6	`== !=`
7	`and`
8	`or`

Example 2.2. Integer division, remainder, and floating-point division

```
imp*ioprt.ed

sta(main){
    iid: sta(int a) => edt: sta(a = 7) ed
    iid: sta(int b) => edt: sta(b = 2) ed
    iid: sta(int q) => edt: sta(q = a / b) ed     \\ q = 3 \\
    iid: sta(int r) => edt: sta(r = a % b) ed     \\ r = 1 \\
    iid: sta(float f) => edt: sta(f = 7.0 / 2.0) ed
    iid: sta(pri) => "q=/d{q} r=/d{r} f=/f{f}" ed
    iid: ref* 0 ed
}
```

Output according to the specification:

```
q=3 r=1 f=3.50
```

Assignment is =, comparison is ==

In the original version, a condition was written as `sta(a = 0)`, using the assignment symbol where a comparison was needed. That assigns 0 to a instead of asking whether a equals 0. Sd 2 makes the distinction explicit: = is for assignment, == is for comparison, and using = in a condition is error E303.

2.5 Variable scope

A variable lives in the nearest enclosing `{ ... }` frame and disappears when that frame closes. A variable declared in the `for` loop header lives only inside that loop. Using a variable outside its scope causes error E301 (undeclared variable).

Chapter summary

Declaration: `iid: sta(type name) ed`; to provide an initial value, add `=> edt: sta(name = value)`. The four basic types are `int`, `float`, `str`, and `bool`. = assigns; == compares. Dividing two integers produces an integer result.

Exercises

2.1 Evaluate: `17 / 5`, `17 % 5`, `17 / 5.0`, `2 + 3 * 4`, `(2 + 3) * 4`.
2.2 Write a program that declares two integers `a = 12` and `b = 5`, then prints their sum,

difference, product, integer quotient, and remainder.
2.3 Write a program that calculates the circumference and area of a circle with radius `r = 2.5`, using `pi = 3.14159`.
2.4 Explain why `sta(a = 0)` is prohibited as a condition in Sd 2.

Chapter 3 — Input, Output, and Formatting

3.1 The `pri` print instruction

A print instruction has the form `iid: sta(pri) => EXPRESSION ed`. Each `pri` instruction automatically moves to a new line after printing. Printing the empty string `""` produces a blank line.

3.2 Fast formatting

Instead of concatenating strings with the plus sign, Sd inserts values directly into a string using formatting placeholders:

Placeholder	Inserts	Example
`/s{x}`	A `str` string	`/s{ten}`
`/d{x}`	An `int` integer	`/d{a + b}`
`/f{x}`	A floating-point number with 2 decimal places	`/f{diem}`
`/f3{x}`	A floating-point number with 3 decimal places [PR]	`/f3{pi}`
`/b{x}`	A `bool` value [PR]	`/b{xong}`
`//`	A literal / character	`"1//2"`

Inside `{ }`, you may put a variable name, an array element, a structure field, or an entire expression.

Curly braces are an improvement [PR]

The original version used an unbraced form such as `/dtemp`. This becomes ambiguous when the variable name touches the following text: does `/dtempC` mean variable `tempC`, or variable `temp` followed by the letter `C`? Curly braces make the boundary explicit. The Sd++ compiler can still read the old unbraced form, but issues a warning.

3.3 The `ipt` input instruction

Read a value from the keyboard into an existing variable with `iid: sta(ipt) => edt: sta(name) ed`. The input data type is determined by the variable's type.

Example 3.1. Read two numbers and print their sum

```
imp*ioprt.ed

sta(main){
    iid: sta(int a) ed
    iid: sta(int b) ed
    iid: sta(pri) => "Enter a:" ed
    iid: sta(ipt) => edt: sta(a) ed
    iid: sta(pri) => "Enter b:" ed
    iid: sta(ipt) => edt: sta(b) ed
    iid: sta(pri) => "/d{a} + /d{b} = /d{a + b}" ed
    iid: ref* 0 ed
}
```

If the user enters a value of the wrong type (for example, text for an `int` variable), the program stops with error E403 [PR]. Chapter 9 explains how to handle this situation.

3.4 Sample program: temperature conversion

Example 3.2. Convert Celsius to Fahrenheit

```
imp*ioprt.ed

sta(main){
    iid: sta(float c) ed
    iid: sta(float f) ed
    iid: sta(pri) => "Enter degrees C:" ed
    iid: sta(ipt) => edt: sta(c) ed
    iid: sta(f = c * 9.0 / 5.0 + 32.0) ed
    iid: sta(pri) => "/f{c} degrees C = /f{f} degrees F" ed
    iid: ref* 0 ed
}
```

The line `iid: sta(f = c * 9.0 / 5.0 + 32.0) ed` is a shortened assignment [PR]: when the instruction header is an assignment to an existing variable, the longer `=> edt:` form is unnecessary. These two forms are equivalent:

```
iid: sta(f) => edt: sta(f = c * 9.0 / 5.0 + 32.0) ed   \\ full form \\
iid: sta(f = c * 9.0 / 5.0 + 32.0) ed                  \\ shortened form \\
```

Chapter summary

`pri` prints and adds a newline; `ipt` reads input according to the variable's type. The `/s{ }`, `/d{ }`, `/f{ }`, and `/b{ }` placeholders insert values into a string; `//` prints a slash.

Exercises

3.1 Write a program that reads a name and age, then prints: `Hello An, next year you will be 20`, where 20 is the age plus one.
3.2 Read two floating-point numbers representing a rectangle's length and width; print its perimeter and area to two decimal places.
3.3 Read a three-digit integer n; print its hundreds, tens, and ones digits (using `/` and `%`).
3.4 Print exactly this line: `Ratio: 3/4`. Write the appropriate `pri` instruction.

Chapter 4 — Branching: The `bolean` Block

4.1 When do we need branching?

Programs often have to make decisions: a sufficiently high score means passing, and a correct password grants access. Sd handles this with a `bolean` block, a "true/false logical structure used to test conditions." The block has two specification tags: `cdn` (condition) and `tf` (true-false, the two true or false branches); it closes with `bln ed`.

The `bolean` block pattern

```
iid: sta(bolean) => cdn,tf:
    sta(CONDITION-1) {
        ...
    }
    sta(CONDITION-2) {          \\ there may be multiple branches \\
        ...
    }
    sta(else) {                 \\ optional: used if no condition is true \\
        ...
    }
bln ed
```

Branches are checked from top to bottom. The first branch whose condition is true is executed; later branches are skipped. If present, `sta(else)` [PR] must be the last branch (otherwise error E306 is reported).

Example 4.1. Grade classification

```
imp*ioprt.ed

sta(main){
    iid: sta(float diem) ed
    iid: sta(pri) => "Enter score:" ed
    iid: sta(ipt) => edt: sta(diem) ed
    iid: sta(bolean) => cdn,tf:
        sta(diem >= 8.0) {
            iid: sta(pri) => "Excellent" ed
        }
        sta(diem >= 6.5) {
            iid: sta(pri) => "Good" ed
        }
        sta(diem >= 5.0) {
            iid: sta(pri) => "Average" ed
        }
        sta(else) {
            iid: sta(pri) => "Poor" ed
        }
    bln ed
    iid: ref* 0 ed
}
```

For `diem = 7.0`, the first branch is false and the second is true, so `Good` is printed; the remaining two branches are not considered.

4.2 `bool` variables and logical operators

The result of a comparison is a `bool` value. You can store it in a variable and reuse it:

Example 4.2. Boolean variables and `and`, `or`, `not`

```
imp*ioprt.ed

sta(main){
    iid: sta(int tuoi) => edt: sta(tuoi = 17) ed
    iid: sta(bool nguoi_lon) => edt: sta(nguoi_lon = tuoi >= 18) ed
    iid: sta(bool hoc_sinh) => edt: sta(hoc_sinh = true) ed
    iid: sta(bool giam_gia) => edt: sta(giam_gia = hoc_sinh or tuoi < 12) ed
    iid: sta(pri) => "adult: /b{nguoi_lon}" ed
    iid: sta(pri) => "discount: /b{giam_gia}" ed
    iid: ref* 0 ed
}
```

4.3 Nested blocks

A `bolean` block may be placed inside one of its branches. Each block closes with its own `bln ed`.

Example 4.3. Even, odd, negative, and positive numbers

```
imp*ioprt.ed

sta(main){
    iid: sta(int n) => edt: sta(n = -4) ed
    iid: sta(bolean) => cdn,tf:
        sta(n > 0) {
            iid: sta(pri) => "positive" ed
```

```
        }
        sta(n < 0) {
            iid: sta(pri) => "negative" ed
            iid: sta(bolean) => cdn,tf:
                sta(n % 2 == 0) {
                    iid: sta(pri) => "and even" ed
                }
            bln ed
        }
        sta(else) {
            iid: sta(pri) => "zero" ed
        }
    bln ed
    iid: ref* 0 ed
}
```

Common mistakes

- Forgetting `bln ed` at the end of a block: error E202 (block not closed).
- Closing with the wrong type, such as using `dbl ed` instead of `bln ed`: error E202.
- Putting branches in the wrong order. For example, if `diem >= 5.0` is checked before `diem >= 8.0`, a score of 9 will enter the "average" branch because that condition is true first. Always check narrower conditions before broader ones.

Chapter summary

Branching uses `iid: sta(bolean) => cdn,tf:`, branches of the form `sta(condition) { ... }`, an optional final `sta(else)` branch, and the closing `bln ed`. Only one branch runs: the first true branch from top to bottom.

Exercises

4.1 Read an integer and print `even` or `odd`.
4.2 Read three integers and print the largest.
4.3 Read a year and print `leap year` if it is divisible by 400, or divisible by 4 but not by 100; otherwise print `not a leap year`.
4.4 Read coefficients `a` and `b` of the equation `a*x + b = 0`; print the solution, `no solution`, or `infinitely many solutions` (pay attention to the cases where `a = 0`).

Chapter 5 — Loops: `for` and `while`

5.1 Two loop families, one `dbl` terminator

Loops belong to the paired family: a block with two specification tags that closes with `dbl ed` (`double`). The word `dbl` appears in two places: when opening the loop body (`dbl sta(...) { ... }`) and when closing the block (`dbl ed`).

Loop	Two tags	Meaning of the two tags
`for`	`cdn,chck`	Stop condition (`cdn`) and step control (`chck`)
`while`	`cdn,edt`	Condition (`cdn`) and a body that modifies variables so the condition can change (`edt`)

5.2 The `for` loop: when the number of iterations is known

The `for` loop pattern

```
iid: sta(for) => cdn,chck:
    dbl sta(INITIALIZATION ' CONDITION ' STEP) {
```

```
        ...loop body...
    }
dbl ed
```

The contents of `sta( )` have three items separated by single quotes: initialization, condition, and step. A variable declared in the initialization item lives only inside the loop. The step may be `i++`, `i--`, or an assignment such as `i = i + 2`. Omitting any of the three items causes error E206.

Example 5.1. Sum from 1 to 10

```
imp*ioprt.ed

sta(main){
    iid: sta(int tong) => edt: sta(tong = 0) ed
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 1 ' i <= 10 ' i++) {
            iid: sta(tong = tong + i) ed
        }
    dbl ed
    iid: sta(pri) => "Sum = /d{tong}" ed
    iid: ref* 0 ed
}
```

Result: `Sum = 55`.

Why must a `for` loop header contain all three items?

The original version wrote `sta(i ' i < 10 ' i++)` without specifying where `i` starts. If `i` has not been declared, the compiler cannot guess; if it has already been declared, the second run may begin from its old value. All three items make the loop produce the same result on every run.

5.3 The `while` loop: repeat according to a condition

The `while` loop pattern

```
iid: sta(while) => cdn,edt:
    dbl sta(CONDITION) {
        ...body: MUST modify at least one variable used in CONDITION...
    }
dbl ed
```

The `edt` tag reminds you of an important logical rule: if the body of a `while` loop never modifies any variable in its condition, the condition never changes and the loop runs forever. The Sd 2 compiler detects this and issues warning W201 [PR].

Example 5.2. Sum of the digits of a number

```
imp*ioprt.ed

sta(main){
    iid: sta(int n) => edt: sta(n = 12345) ed
    iid: sta(int tong) => edt: sta(tong = 0) ed
    iid: sta(while) => cdn,edt:
        dbl sta(n > 0) {
            iid: sta(tong = tong + n % 10) ed
            iid: sta(n = n / 10) ed
        }
    dbl ed
    iid: sta(pri) => "Sum of digits = /d{tong}" ed
    iid: ref* 0 ed
}
```

Result: `Sum of digits = 15.`

5.4 Stop early and skip: `brk`, `nxt`

- `iid: brk ed` [PR] exits the nearest enclosing loop completely.
- `iid: nxt ed` [PR] skips the rest of the current iteration and moves to the next one.

Example 5.3. The first multiple of 7 greater than 100

```
imp*ioprt.ed

sta(main){
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 101 ' i <= 200 ' i++) {
            iid: sta(bolean) => cdn,tf:
                sta(i % 7 == 0) {
                    iid: sta(pri) => "First multiple of 7: /d{i}" ed
                    iid: brk ed
                }
            bln ed
        }
    dbl ed
    iid: ref* 0 ed
}
```

Result: `First multiple of 7: 105.`

5.5 Nested loops

Example 5.4. Multiplication tables from 2 to 4

```
imp*ioprt.ed

sta(main){
    iid: sta(for) => cdn,chck:
        dbl sta(int a = 2 ' a <= 4 ' a++) {
            iid: sta(for) => cdn,chck:
                dbl sta(int b = 1 ' b <= 3 ' b++) {
                    iid: sta(pri) => "/d{a} x /d{b} = /d{a * b}" ed
                }
            dbl ed
        }
    dbl ed
    iid: ref* 0 ed
}
```

Each `for` loop closes with its own `dbl ed`; the inner loop closes before the outer loop.

Common mistakes

- A `while` loop does not modify its condition variable (warning W201; the loop runs forever).
- An off-by-one error: writing `i < 10` when `i <= 10` is required.
- Closing the wrong block: if the inner loop is missing its `dbl ed`, the outer loop effectively loses its terminator.

Chapter summary

Loops are paired-family blocks and close with `dbl ed`. A `for` loop has three single-quote-separated items: initialization, condition, and step. A `while` loop has one condition, and its body must modify a variable in that condition. `brk` exits; `nxt` skips the current iteration.

Exercises

5.1 Print the even numbers from 2 to 20, one per line.
5.2 Read n and calculate `n!` (the product from 1 through n) using a `for` loop.
5.3 Read n and count its digits using a `while` loop.
5.4 Print a four-row star triangle (row k contains k asterisks). Hint: use two nested `for` loops and a `str` variable assembled with the `str` library (Chapter 6).
5.5 Find and fix the error: a `while` loop has condition `n != 0`, but its body contains only a print instruction.

Chapter 6 — Arrays and Strings

6.1 Arrays: multiple values of the same type

An array is a sequence of contiguous memory cells of the same type, indexed starting at 0. Declare its size in square brackets. Elements in an initial-value list are separated by single quotes `'` (the same separator used in a `for` loop header).

Example 6.1. Declare an array, read and write an element

```
imp*ioprt.ed

sta(main){
    iid: sta(int a[5]) => edt: sta(a = [4 ' 9 ' 1 ' 7 ' 3]) ed
    iid: sta(a[0] = 10) ed
    iid: sta(pri) => "a[0] = /d{a[0]}, a[4] = /d{a[4]}" ed
    iid: ref* 0 ed
}
```

Commas and single quotes

Sd uses two separators for two distinct purposes; they are never interchangeable:

- A comma `,` separates specification tags at the beginning of a block: `cdn,chck`.
- A single quote `'` separates data items: array elements, function arguments, items in a `for` header, and structure fields.

A valid index ranges from 0 to the array size minus one. Accessing an out-of-range index at runtime causes error E404.

6.1.1 Traversing an array and finding the maximum

Example 6.2. Find the largest element

```
imp*ioprt.ed

sta(main){
    iid: sta(int a[5]) => edt: sta(a = [4 ' 9 ' 1 ' 7 ' 3]) ed
    iid: sta(int mx) => edt: sta(mx = a[0]) ed
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 1 ' i < 5 ' i++) {
            iid: sta(bolean) => cdn,tf:
                sta(a[i] > mx) {
                    iid: sta(mx = a[i]) ed
                }
            bln ed
        }
    dbl ed
    iid: sta(pri) => "Max = /d{mx}" ed
    iid: ref* 0 ed
}
```

Result: `Max = 9`.

6.1.2 Bubble sort

Example 6.3. Sort in ascending order

```
imp*ioprt.ed

sta(main){
    iid: sta(int a[5]) => edt: sta(a = [5 ' 2 ' 9 ' 1 ' 7]) ed
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 0 ' i < 4 ' i++) {
            iid: sta(for) => cdn,chck:
                dbl sta(int j = 0 ' j < 4 - i ' j++) {
                    iid: sta(bolean) => cdn,tf:
                        sta(a[j] > a[j + 1]) {
                            iid: sta(int t) => edt: sta(t = a[j]) ed
                            iid: sta(a[j] = a[j + 1]) ed
                            iid: sta(a[j + 1] = t) ed
                        }
                    bln ed
                }
            dbl ed
        }
    dbl ed
    iid: sta(for) => cdn,chck:
        dbl sta(int k = 0 ' k < 5 ' k++) {
            iid: sta(pri) => "a[/d{k}] = /d{a[k]}" ed
        }
    dbl ed
    iid: ref* 0 ed
}
```

Result: `a[0] = 1`, `a[1] = 2`, `a[2] = 5`, `a[3] = 7`, `a[4] = 9`, with each value on its own line.

6.2 Strings

A `str` string is a sequence of characters enclosed in double quotes. String operations are in the `str` library (loaded with `imp*str.ed`):

Function	Result	Example
`str.len(s)`	Length (number of characters)	`str.len("Sd++") → 4`
`str.upper(s)`	Uppercase string	`str.upper("sd") → "SD"`
`str.lower(s)`	Lowercase string	`str.lower("SD") → "sd"`
`str.concat(a ' b)`	Concatenate two strings	`str.concat("Sd" ' "++") → "Sd++"`

Two strings are compared using == and `!=`.

Example 6.4. String operations

```
imp*ioprt.ed
imp*str.ed

sta(main){
    iid: sta(str ten) => edt: sta(ten = "Sd++") ed
    iid: sta(int n) => edt: sta(n = str.len(ten)) ed
    iid: sta(str hoa) => edt: sta(hoa = str.upper("sd lang")) ed
    iid: sta(str ghep) => edt: sta(ghep = str.concat(ten ' "!")) ed
    iid: sta(pri) => "/s{ten} has /d{n} characters" ed
    iid: sta(pri) => "/s{hoa} and /s{ghep}" ed
```

```
    iid: ref* 0 ed
}
```

Result:

```
Sd++ has 4 characters
SD LANG and Sd++!
```

Chapter summary

Array: `sta(type name[N])`, indices start at 0, and initial values use `[x ' y ' z]`. String type: `str`, with operations through the `str` library. A comma separates tags; a single quote separates data.

Exercises

6.1 Given the array `[3 ' 8 ' 2 ' 6 ' 1]`, print its sum and arithmetic mean (floating-point, two decimal places).
6.2 Count how many elements in an array are even.
6.3 Reverse a five-element array (without using a second array).
6.4 Print a four-row star triangle by building a string `dong` and appending one asterisk after each iteration.
6.5 Read a string and print `empty` if its length is 0; otherwise print its length.

Chapter 7 — Functions and Procedures: `fn` and `void`

7.1 Why divide a program into smaller parts?

As a program grows, repeated tasks can be separated into named functions and called whenever needed. The original version describes this as a part for “freely creating and inventing callable functions”: the programmer chooses the name and decides what the function does. Sd divides these into two families according to whether they return a value.

Family	Frame	Tag	Closer
Function (returns a value)	`iid: sta(fn) => rtn:`	`rtn` (return)	`fn ed`
Procedure (returns no value)	`iid: sta(void) => unkw:`	`unkw` (unknown)	`void ed`

7.2 Functions that return a value: `fn`

Example 7.1. A function that adds two numbers

```
imp*ioprt.ed

iid: sta(fn) => rtn:
    sta cong(int a ' int b) -> int {
        iid: ref* a + b ed
    }
fn ed

sta(main){
    iid: sta(int s) => edt: sta(s = cong(3 ' 4)) ed
    iid: sta(pri) => "3 + 4 = /d{s}" ed
    iid: ref* 0 ed
}
```

Function rules

1. Write the function name, parameter list (`type name ' type name`), and then `-> return-type`.

2. Parameters are separated by `'`.
3. The function body ends with an outermost instruction of the form `iid: ref* VALUE ed` (error E207 if it is missing).
4. Functions are placed outside `sta(main)`; the order of top-level blocks does not matter [PR], so a function placed after `main` can still be called.
5. Numeric, string, and Boolean parameters are passed by value (the function receives a copy). Arrays are passed by address [PR]: modifying an element inside the function changes the caller's array.

7.2.1 Recursion

A function is allowed to call itself. Every recursive function needs a stopping point so that the chain of calls can end.

Example 7.2. Factorial by recursion

```
imp*ioprt.ed

iid: sta(fn) => rtn:
    sta giaithua(int n) -> int {
        iid: sta(bolean) => cdn,tf:
            sta(n <= 1) {
                iid: ref* 1 ed
            }
        bln ed
        iid: ref* n * giaithua(n - 1) ed
    }
fn ed

sta(main){
    iid: sta(pri) => "5! = /d{giaithua(5)}" ed
    iid: ref* 0 ed
}
```

Result: `5! = 120`. The `n <= 1` branch is the stopping point.

7.2.2 Passing arrays

Example 7.3. A function that sums an array

```
imp*ioprt.ed

iid: sta(fn) => rtn:
    sta tong(int a[5]) -> int {
        iid: sta(int s) => edt: sta(s = 0) ed
        iid: sta(for) => cdn,chck:
            dbl sta(int i = 0 ' i < 5 ' i++) {
                iid: sta(s = s + a[i]) ed
            }
        dbl ed
        iid: ref* s ed
    }
fn ed

sta(main){
    iid: sta(int d[5]) => edt: sta(d = [1 ' 2 ' 3 ' 4 ' 5]) ed
    iid: sta(pri) => "Sum = /d{tong(d)}" ed
    iid: ref* 0 ed
}
```

7.3 Procedures: `void`

A procedure performs a task but does not return a value. The original version describes a `void` block as “a type outside the data types, which is not read unless called.” This means that a procedure’s body runs only when there is a `call` instruction; merely declaring it does nothing.

Example 7.4. A greeting procedure and the `call` instruction

```
imp*ioprt.ed

iid: sta(void) => unkw:
    sta chaomung(str ten) {
        iid: sta(pri) => "Hello, /s{ten}!" ed
    }
void ed

sta(main){
    iid: sta(pri) => "Start" ed
    iid: call chaomung("An") ed
    iid: call chaomung("Binh") ed
    iid: ref* 0 ed
}
```

Result:

```
Start
Hello, An!
Hello, Binh!
```

`call` and calls inside expressions [PR]

A procedure is called as an instruction: `iid: call name(...) ed`. A function that returns a value is used directly in an expression, such as `cong(3 ' 4)`. If you want to call a function but discard its result, you can also write `iid: call name(...) ed`.

Chapter summary

Function: `iid: sta(fn) => rtn: ... fn ed`, with `-> type` and a final `ref*`. Procedure: `iid: sta(void) => unkw: ... void ed`, called with `call`. Parameters are separated by `'`.

Exercises

7.1 Write a function `lonnhat(int a ' int b) -> int` and use it to find the largest of three numbers.
7.2 Write a function `lachinhphuong(int n) -> bool`, then print the perfect squares from 1 to 50.
7.3 Write a procedure `invach(int n)` that prints a line containing `n` hyphens.
7.4 Write a recursive function `fib(int n) -> int` that returns the nth Fibonacci number (`fib(0)=0`, `fib(1)=1`).

Chapter 8 — Data Structures: `struct`

8.1 Combining multiple pieces of information into one

A student has a name, age, and score. Instead of using three separate variables, we group them into one structure. The `struct` block belongs to the single-tag family: it has only one `dta` (data) tag and closes with `strt ed`.

Example 8.1. Declaring and using a structure

```
imp*ioprt.ed

iid: sta(struct) => dta:
    sta SinhVien { str ten ' int tuoi ' float diem }
```

```
strt ed

sta(main){
    iid: sta(SinhVien sv) ed
    iid: sta(sv.ten = "An") ed
    iid: sta(sv.tuoi = 19) ed
    iid: sta(sv.diem = 8.5) ed
    iid: sta(pri) => "/s{sv.ten}, age /d{sv.tuoi}, score /f{sv.diem}" ed
    iid: ref* 0 ed
}
```

Result: `An, age 19, score 8.50`.

Structure rules

1. Fields are separated by `'`; each field is `type name`. Field names must be unique (error E302).
2. Use the structure name as a type: `sta(SinhVien sv)`.
3. Access a field with a dot: `sv.ten`.
4. Assigning one structure to another structure of the same type copies all fields [PR].

Structures are different from arrays

The original version used `sta arr[int a(10) ' str("hello world") ' float a(1.4)]`, mixing the array pattern with the structure pattern and using the field name a twice. Sd 2 separates them clearly: `[ ]` is used only for arrays (elements of the same type), while `{ }` encloses the different-typed fields of a structure.

8.2 Arrays of structures

Example 8.2. Find the student with the highest score

```
imp*ioprt.ed

iid: sta(struct) => dta:
    sta SinhVien { str ten ' float diem }
strt ed

sta(main){
    iid: sta(SinhVien ds[3]) ed
    iid: sta(ds[0].ten = "An") ed
    iid: sta(ds[0].diem = 7.0) ed
    iid: sta(ds[1].ten = "Binh") ed
    iid: sta(ds[1].diem = 9.0) ed
    iid: sta(ds[2].ten = "Chi") ed
    iid: sta(ds[2].diem = 8.0) ed
    iid: sta(int vt) => edt: sta(vt = 0) ed
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 1 ' i < 3 ' i++) {
            iid: sta(bolean) => cdn,tf:
                sta(ds[i].diem > ds[vt].diem) {
                    iid: sta(vt = i) ed
                }
            bln ed
        }
    dbl ed
    iid: sta(pri) => "Highest: /s{ds[vt].ten}" ed
    iid: ref* 0 ed
}
```

Result: `Highest: Binh`.

8.3 Structures and functions

A structure can be used as a function parameter or return type (passed by value, meaning that a copy is made).

Example 8.3. A function that accepts a structure

```
imp*ioprt.ed

iid: sta(struct) => dta:
    sta Diem { int x ' int y }
strt ed

iid: sta(fn) => rtn:
    sta tong(Diem p) -> int {
        iid: ref* p.x + p.y ed
    }
fn ed

sta(main){
    iid: sta(Diem p) ed
    iid: sta(p.x = 3) ed
    iid: sta(p.y = 4) ed
    iid: sta(pri) => "x + y = /d{tong(p)}" ed
    iid: ref* 0 ed
}
```

Chapter summary

`struct` is a single-family block: it uses the `dta` tag and closes with `strt ed`. Fields are separated by `'` and must have unique names. Access fields with a dot; arrays of structures are allowed.

Exercises

8.1 Define a PhanSo structure containing `tu` and `mau` (integers). Write a function that adds two fractions (without simplifying the result).
8.2 Create an array of three `SinhVien` records, read their names and scores from the keyboard, and print the class average.
8.3 Find the error in the declaration `sta Xe { str hang ' int doi ' str hang }`.
8.4 Why is `SinhVien.ten = "An"` (using the type name instead of a variable name) incorrect?

Chapter 9 — Error Codes and Debugging

9.1 Three kinds of problems

A program can go wrong in three different ways, and each requires a different response.

Type	When it occurs	Code
Syntax or semantic error	During compilation, before the program runs	E1xx, E2xx, E3xx
Runtime error	The program encounters a prohibited situation while running	E4xx
Warning	The code is valid but suspicious; it can still run	Wxxx

The complete code table is in Appendix C.

9.2 Error codes

The original version has a note beside the `ref*` instruction: when code runs with incorrect logic, it “runs out an error-point value.” Sd 2 turns this idea into a convention:

Error-code convention [PR]

- `ref* 0` means success.
- `ref* N`, where N is from 1 to 254, is an error code chosen by the program itself.
- When a runtime error occurs, the system prints `[ERROR Exxx @line N]` with a description, then terminates with code 255.

The most common way to protect a program is to validate data immediately after input and exit using your own error code:

Example 9.1. Validate input and return an error code

```
imp*ioprt.ed

sta(main){
    iid: sta(int tuoi) ed
    iid: sta(pri) => "Enter age:" ed
    iid: sta(ipt) => edt: sta(tuoi) ed
    iid: sta(bolean) => cdn,tf:
        sta(tuoi < 0 or tuoi > 150) {
            iid: sta(pri) => "Invalid age" ed
            iid: ref* 2 ed
        }
    bln ed
    iid: sta(pri) => "Valid age: /d{tuoi}" ed
    iid: ref* 0 ed
}
```

9.3 Reading an error message

An error message has four parts: type and error code, line where it occurred, description, and suggestion.

Example error message

```
[ERROR E301 @line 3] variable 'b' is undeclared
    iid: sta(pri) => "a = /d{b}" ed
    suggestion: did you mean 'a'?
```

9.4 Practice: find the errors in the following program

Example 9.2. A program with three problems (for debugging practice)

```
imp*ioprt.ed

sta(main){
    iid: sta(int a) => edt: sta(a = 3) ed
    iid: sta(pri) => "a = /d{b}" ed
    iid: sta(while) => cdn,edt:
        dbl sta(a > 0) {
            iid: sta(pri) => "loop" ed
        }
    dbl ed
}
```

The three problems are: (1) E301, variable `b` is used without being declared; (2) W201, the `while` loop does not modify `a`, so it runs forever; (3) E204, the `main` frame is missing `iid: ref* ...` ed at the end.

9.5 Five-step debugging process

1. Read the error code and look it up in Appendix C; do not guess.
2. Go to the reported line, but also inspect the line before it (a missing terminator is often reported on the following line).
3. Narrow down the problem: insert `pri` instructions to print variable values and see how far the program gets.
4. Fix one thing and run again; if you fix several things at once, you will not know which change helped.
5. Record the cause so you do not repeat the same mistake next time.

Pre-run checklist

- Every one-line `iid:` instruction ends with `ed`.
- Every opened block has the correct closing terminator: `dbl`, `strt`, `void`, `bln`, or `fn`.
- The number of `{` braces equals the number of `}` braces.
- Every opened `\\` comment has a closing `\\`.
- `main` ends with `ref*`; a function that returns a value also ends with `ref*`.

Chapter summary

Errors fall into three categories: compilation errors, runtime errors, and warnings. `ref* 0` means success; `ref* N` is an error code chosen by you; a runtime error returns 255. Debug in five steps: read the code, go to the line, narrow down the problem, fix one thing, and record the cause.

Exercises

9.1 A program is missing the `ed` terminator on a `pri` instruction. Which error code do you predict? Check Appendix C.
9.2 Write a program that reads two numbers and prints their quotient; if the divisor is 0, print `division by zero` and terminate with `ref* 3`.
9.3 Given an array `a[5]`, what runtime error is caused by `sta(pri) => "/d{a[5]}"`? Why?
9.4 Rewrite the following infinite loop using Sd 2 syntax and make it terminate: `n = 10; while n != 0 { pri n }`.

Part II — Advanced Sd++

Chapter 10 — Sd++ and the `ssta` / ## Boundary

10.1 What is Sd++?

Sd++ is an extended version of Sd for systems programming. Everything you learned in Part I (the `sta` frame, `iid` instructions, blocks, functions, and structures) remains fully usable in Sd++. Sd++ adds four things:

1. **Absolute boundary:** `ssta` at the beginning and ## at the end.
2. **Terminator-flag set:** $, ₫, €, $$, and # for different kinds of scope.
3. **Low-level memory:** pointers @, *, and Heap allocation.
4. **Metaprogramming:** `define` for renaming, `defk` for creating custom terminator flags, and the self-checking selection block `ids sb` with Reback.

Relationship between Sd and Sd++ [PR]

A valid Sd program becomes a valid Sd++ program when it is wrapped in `ssta { ... }` and ends with ##. In Sd++, ed is understood as the $ flag at the instruction level. In other words, you do not have to relearn anything; Part II only adds tools.

10.2 The security box

Every Sd++ program is like a box. `ssta` (Super Statement) is the opening lid, placed at the beginning of the file; it establishes semantics and memory for the entire program. ## is the closing lid, placed at the end of the file: when the compiler or runtime reaches ##, the entire Call Stack is destroyed, registers are cleared, Heap and Stack memory are released, and the machine returns to its initial state. This closing lid ensures that even if a program encounters a severe error, resources are not left behind.

Example 10.1. Minimal frame of an Sd++ program

```
ssta {
    */ all source code goes here
}
##
```

Nothing after

must be the final code in the file; only whitespace and comments may follow it. Omitting ## causes error E208. ## makes it possible to terminate the entire program without counting how many unclosed braces remain.

10.3 The terminator-flag set

In Sd++, the central principle is "use the correct flag for the correct scope." Do not scatter terminator symbols everywhere; terminate only where an instruction or scope actually ends.

Flag	Name	Purpose
$	Instruction-scope terminator	Terminates an executable instruction inside a frame
đ	Bare global terminator	Terminates a global variable declaration outside every frame
€	Outer-frame terminator	Terminates an entire `define[...]{ ... }` block; uses the "close first, load definitions later" mechanism
$$	Conditional jump	Inside `ids sb`: Reback, return to check conditions again; outside the block: emergency stop
#	Segment station	Seals a `defk` block; an anchor point for recovery after an error
##	Terminate all	Closes the entire program and cleans up resources

One terminator per scope

Each terminator flag appears only once for a given instruction or scope. A global declaration line ends with exactly one đ at the end of the line; a `define` block ends with exactly one € at the end of the block.

$$ outside an sb block is an emergency stop

When `$$` appears outside an `ids sb` block, it becomes a **Hard Halt**: the entire system stops immediately (like a "Kernel Spinlock"). Do not use `$$` outside `ids sb` unless you genuinely intend an emergency stop.

10.4 Comments in Sd++

Sd++ supports both comment styles: `*/` for a single-line comment (from `*/` to the end of the line), and the pair `\\ ... \\` for a multi-line comment. Choose one style per file for consistency.

10.5 Your first Sd++ program

Example 10.2. Hello Sd++

```
ssta {
    imp*ioprt.$

    define[
        ids.edt: text = "main",
        pri.edt: text = "print"
    ]{
        main: {
            ids.pri: "Welcome to Sd++!" $
            ref*0 $:
        } $
    } €
}
##
```

Read it as follows:

- `ssta { ... }` and `##` form the box.
- `imp*ioprt.$` loads the library (the `$` terminator replaces `.ed` in Sd).
- `define[ ... ]{ ... } €` is the outer frame, covered in Chapter 11.
- `ids.pri: "..." $` is the flat-form print instruction, equivalent to `iid: sta(pri) => "..." ed` in Sd.
- `ref*0 $:` ends the `main` frame.

Chapter summary

Sd++ = Sd + the `ssta` / `##` boundary + terminator flags + low-level memory + metaprogramming. `ed` is `$` at the instruction level. `$$` outside `sb` is an emergency stop.

Exercises

10.1 Write an Sd++ program that prints two lines using `ids.pri`.
10.2 Fill in the appropriate terminator flag (`$`, `đ`, `€`): (a) `ids.g: x = 5 ?` (b) `ids.pri: "a" ?` (c) `define[ ... ]{ ... } ?`
10.3 What happens when an Sd++ file is missing `##`?

Chapter 11 — Bare Global Variables and the `define` Frame

11.1 Bare global variables

A variable declared outside every frame is called a **bare global variable** and can be used anywhere in the program. Declare it with `ids.g:` and terminate it with `đ`; access it again with `ids(name).g`.

Example 11.1. Global and local variables

```
ssta {
    imp*ioprt.$

    ids.g: ten_may = "Server A" đ
    ids.g: nhiet_do = -20 đ

    define[
        ids.edt: text = "main",
```

```
        pri.edt: text = "print"
    ]{
        main: {
            iptedt.int t = ids(nhiet_do).g $
            ids.pri: "Machine: /s{ids(ten_may).g}" $
            ids.pri: "Temperature: /d{t}" $
            ref*0 $:
        } $
    } €
}
##
```

The line `iptedt.int t = ids(nhiet_do).g $` declares the local variable `t` (terminated with `$`) and assigns it the value of the global variable. The output according to the specification is:

```
Machine: Server A
Temperature: -20
```

Global-variable rules [PR]

1. A local variable must not have the same name as a global variable (error E302), so that `ids(x).g` and `x` are never ambiguous.
2. A global variable may be a number, string, array (`sta.arr.g`), or pointer (`ptr.g`); all four use the đ terminator.
3. Global arrays separate elements with `'`, as everywhere else: `sta.arr.g: so = [10 ' 20 ' 30] đ`. The older comma-separated form in the original specification is still accepted, with warning W202.

11.2 The `define[...]{ ... } €` frame

The outer frame does two things: it renames standard components (the main program and print instruction) and contains the main program.

Anatomy of the `define` frame

```
define[
    ids.edt: text = "main",         <- assigns the name "main" to the main-program fra
    pri.edt: text = "print"         <- assigns the name "print" to the print instructi
]{
    main: {                         <- main frame; name comes from the line above
        ...
    } $
} €                                 <- closes the outer frame
```

The list inside `[ ]` contains definitions separated by commas. Each definition has the form `type.edt: text = "name"`. The following `{ }` contains the frame body. According to the original examples, after `pri.edt: text = "in_ra"`, you may write `in_ra: "..." $` instead of `ids.pri: "..." $`; the full form `ids.pri:` is always available.

Example 11.2. Rename the print instruction to `in_ra`

```
ssta {
    imp*ioprt.$

    define[
        ids.edt: text = "main",
        pri.edt: text = "in_ra"
    ]{
        main: {
            in_ra: "Use the new name" $
            ids.pri: "Use the full name" $
            ref*0 $:
```

```
        } $
    } €
}
##
```

11.2.1 The “close first, load definitions later” mechanism

The € flag is at the end of the frame, but the definitions in `define[ ]` apply to the entire frame body. Therefore, the compiler makes two passes: pass 1 reads up to € to determine where the frame ends; pass 2 loads the definitions and only then parses the frame body. This lets you use a new name from the very first line of the frame body.

11.3 Comparing flat Sd++ with Sd

Sd (Chapters 1–9)	Flat-form Sd++
`iid: sta(pri) => "x" ed`	`ids.pri: "x" $`
`iid: sta(int a) => edt: sta(a = 5) ed`	`iptedt.int a = 5 $`
`iid: sta(ptr p) => edt: sta(p = @a) ed`	`iptedt.ptr p = @a $`
Declaration outside every frame	`ids.g: a = 5 đ`
`iid: ref* 0 ed`	`ref*0 $:`
ed (instruction terminator)	`$`
`sta(main){ ... }`	`define[...]{ main: { ... } $ } €`

Both forms may appear side by side in the same Sd++ file; choose one style for the entire project to make it easier to read.

Chapter summary

Bare global variable: declare with `ids.g: name = value đ`, and access with `ids(name).g`. The outer frame `define[...]{...} €` renames standard components and contains `main`; the compiler loads definitions in two passes.

Exercises

11.1 Declare two global variables, `ten` and `tuoi`, and print them using the `/s{ }` and `/d{ }` formats.
11.2 Why do global variables end with đ while variables inside a frame end with $?
11.3 Rewrite these using Sd: `iptedt.int a = 5 $` and `ids.pri: "hi" $`.

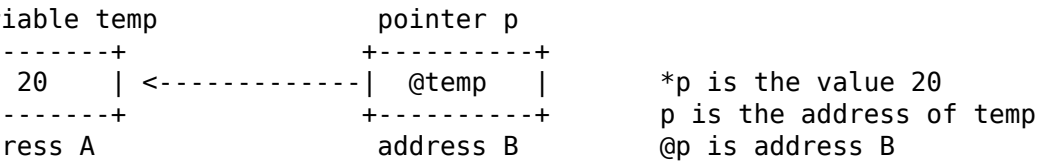
Chapter 12 — Pointers and Heap Memory

12.1 What is a pointer?

Every variable occupies a location in virtual memory called an **address**. A pointer is a variable that stores an address. Sd++ uses two symbols:

- @x (get address): returns the address of variable or array x.
- *p (dereference): reads or writes the value at the address to which p points.

Visualizing memory



Example 12.1. Read and write through a pointer

```
ssta {
    imp*ioprt.$

    define[
        ids.edt: text = "main",
        pri.edt: text = "print"
    ]{
        main: {
            iptedt.int temp = 20 $
            iptedt.ptr p = @temp $
            *p = *p + 5 $
            ids.pri: "temp = /d{temp}" $
            ref*0 $:
        } $
    } €
}
##
```

Result: `temp = 25`. The instruction `*p = *p + 5` modifies the variable `temp` itself because `p` points to it.

Global pointers

A pointer can also be declared outside a frame: `ptr.g: p_nhiet = @nhiet_do đ`. Declare the variable that the pointer points to before declaring the pointer.

12.2 Heap memory

Ordinary variables disappear automatically when their frame closes. When you need a memory region to live longer, or its size is known only at runtime, request memory from the Heap:

- `sta.alloc(n)` allocates n bytes and returns a pointer to the region.
- `sta.free(p)` returns the memory region to the system.

One memory cell is 8 bytes [PR], so `sta.alloc(32)` provides four cells. Cell k, counted from pointer h, is `*(h + k)`; `*h` is the first cell (`k = 0`).

Example 12.2. Allocate, use, and free memory

```
ssta {
    imp*ioprt.$
    imp*mem.$

    define[
        ids.edt: text = "main",
        pri.edt: text = "print"
    ]{
        main: {
            iptedt.ptr h = sta.alloc(32) $
            iid: sta(for) => cdn,chck:
                dbl sta(int i = 0 ' i < 4 ' i++) {
                    iid: sta(*(h + i) = i * i) ed
                }
            dbl ed
            ids.pri: "Cell 3 = /d{*(h + 3)}" $
            sta.free(h) $
            ref*0 $:
        } $
    } €
}
##
```

Result: `Cell 3 = 9`. This example places an Sd instruction (`iid: ... ed`) beside flat-form instructions, as discussed in Chapter 11.

12.3 Memory-safety rules

Four rules

1. **Allocate, then free.** Every `sta.alloc` needs a corresponding `sta.free`; omitting it causes a memory leak (warning W302).
2. After `sta.free(p)`, `p` becomes a null pointer [PR]. Dereferencing a null pointer causes error E405.
3. Freeing a null pointer or freeing the same allocation twice causes error E406.
4. Do not read or write outside the allocated region. For example, `*(h + 4)` when only four cells were requested causes error E408. Requesting too much memory causes E407.

Common mistakes

- Forgetting `sta.free` and then allocating repeatedly in a loop: memory gradually runs out.
- Using a pointer after freeing it.
- Confusing `p` (the address) with `*p` (the value).

Chapter summary

@x gets an address; `*p` reads or writes the value at that address. Heap: `sta.alloc(n)` allocates and `sta.free(p)` frees; every allocation needs a corresponding free. Null pointers and out-of-bounds access are runtime errors.

Exercises

12.1 Write a program that swaps the values of two variables x and y using two pointers.
12.2 Allocate four cells, store 10, 20, 30, and 40, print their sum, then free the memory.
12.3 Find the error: `sta.free(h) $` followed by `*h = 1 $`.

Chapter 13 — The `ids sb` Selection Block and Reback

13.1 The idea: check again automatically

In Part I, `while` is used when you want to repeat a condition check. Sd++ adds another block: the selection block `ids sb(variable)`. Like `bolean`, it has multiple branches, but with one extra ability: after a branch modifies a variable, the block can return to its beginning and recheck the conditions using the new value. This behavior is called **Reback** and is marked by the `$$` flag at the end of a branch.

13.2 Syntax

Selection-block pattern

```
ids sb(VARIABLE) {
    block CONDITION-1:
        ...
        transf $               <- done; exit the block
    block CONDITION-2:
        ...modify VARIABLE...
        $$                     <- Reback: return to the beginning and check again
    remaining:                 <- when no block condition is true
        ...
        break $                <- exit the block immediately
} $
```

Component	Meaning
`ids sb(variable)`	Opens the block; the variable is the value on which the conditions depend.
`block condition:`	A branch that runs if its condition is true.
`remaining:`	The fallback branch, run when none of the `block` conditions is true.
`transf $`	Ends the branch and exits the block (completion).
`break $` [PR]	Exits the block immediately; used in a branch that cannot process anything.
`$$`	Ends the branch and performs Reback: returns to the beginning of the block.

Simplification compared with the original version [PR]

The original version used `$$` for two jobs: Reback in a branch that repairs a value, and "finish" in `break $$`; it also wrote `transf $ $` with two separate terminators. This gave one flag two meanings. Sd++ 2 separates them: `$$` means Reback only; exit the block with `transf $` or `break $`. The old form is still readable, with warning W202.

13.3 Flow diagram

```
Check block branches from top to bottom
                 |
                 v
Run the first branch whose condition is true
(if none is true, run remaining)
                 |
                 v
How does the branch end?
        /                      \
       v                        v
transf $ / break $              $$
   Exit the block          Reback: return to the
                           beginning of the block
```

13.4 Example: self-heating system

Example 13.1. A negative temperature increases until stable

```
ssta {
    imp*ioprt.$

    define[
        ids.edt: text = "main",
        pri.edt: text = "print"
    ]{
        main: {
            iptedt.int temp = -25 $
            ids.pri: "Initial temperature: /d{temp}" $
            ids sb(temp) {
                block temp > 0:
                    ids.pri: "Stable: /d{temp}" $
                    transf $
                block temp < 0:
                    ids.pri: "Negative (/d{temp}), heating" $
                    temp = temp + 20 $
                    $$
```

```
                remaining:
                    ids.pri: "Equal to 0; set to 1" $
                    temp = 1 $
                    $$
            } $
            ref*0 $:
        } $
    } €
}
##
```

Trace each step with `temp = -25`:

Iteration	Selected branch	Result
1	`temp < 0`	Prints `Negative (-25)...`; `temp` becomes -5; Reback
2	`temp < 0`	Prints `Negative (-5)...`; `temp` becomes 15; Reback
3	`temp > 0`	Prints `Stable: 15`; `transf` exits the block

13.5 Preventing infinite loops

Like `while`, Reback can repeat forever. Sd++ 2 adds two protection layers [PR]:

Reback safety

1. **Compile-time check.** A branch ending with `$$` must modify the `sb` variable (directly or through a pointer to that variable). If the compiler sees no sign of such a modification, it issues warning W203.
2. **Runtime limit.** Each activation of an `sb` block allows at most 65,536 Reback operations. Exceeding that limit causes error E402.

13.6 Comparison with `while`

The `sb` block above is equivalent to a `while` loop containing branching. Choose `sb` when each value of a variable needs its own handling and that handling may move the variable into a better state; choose `while` when a simple repetition is all that is needed.

Chapter summary

`ids sb(variable)` has `block condition:` branches and a `remaining:` branch. A branch ends with `transf $` (complete), `break $` (exit), or `$$` (Reback, check again). Infinite repetition is guarded by W203 and E402.

Exercises

13.1 Write an `ids sb` block for `score = -15`: if the score is nonpositive, add 30 and Reback; when it is positive, print `Score review successful`.
13.2 Rewrite Exercise 13.1 using an Sd `while` loop.
13.3 Why is a branch that ends with `$$` without modifying the `sb` variable dangerous?

Chapter 14 — Metaprogramming: `defk` and Custom Terminator Flags

14.1 Creating a custom terminator flag

The `$`, `€`, and `$$` flags are sufficient for most cases, but sometimes you may want a symbol of your own for a project. The `defk` (define key) block lets you create a new terminator flag, such

as ¥, and seal the rule with # so nobody can overwrite it.

Example 14.1. Create and use the ¥ terminator flag

```
ssta {
    imp*ioprt.$

    defk<ids.end: text "¥"> {
        init ids.edt ("¥" endlogic == "$"+"€"+"$$")
        func * end.sta * end.iid #
    }

    define[
        ids.edt: text = "main",
        pri.edt: text = "print"
    ]{
        main: {
            init ¥
            ids.pri: "Use the custom terminator" ¥
            ref*0 ¥:
        } ¥
    } €
}
##
```

14.2 Reading each part of `defk`

This section is the compiler's interpretation based on the original description; the author may revise it.

Component	Meaning
`defk<ids.end: text "¥"> {`	Opens the block and declares ¥ as a new terminator symbol.
`init ids.edt (...)`	Activation condition: ¥ carries the ending logic formed by the union of $, €, and $$; wherever one of these three flags is required, ¥ may stand in for it.
`func * end.sta * end.iid`	Applies to every ending of a `sta` frame and an `iid` instruction (the * means "all").
`#`	Seals the newly declared rule so no later `define` or `init` can overwrite it.
`init ¥`	Activates ¥ in the current frame.

14.3 Rules for `defk`

Four rules

1. A `defk` block must end with #; omitting it causes error E210.
2. Reserved flags $, ₫, €, $$, #, and ## must not be redefined, and a sealed flag must not be overwritten; violations cause error E209 [PR].
3. `defk` must appear before the first use of the new flag; inside a frame, `init ¥` must appear before ¥ is used.
4. A custom flag should be a character that does not normally appear in code, so the reader does not confuse it with a string or operator.

The role of #: an anchor point

In addition to sealing `defk`, # is a recovery anchor: when the parser encounters a syntax error, it skips the damaged section until the nearest #, then resumes in a clean state (Panic Mode

Recovery, Chapter 16). Thus, # helps prevent one error from producing dozens of false error messages.

Chapter summary

`defk<...> { init ... func ... # }` creates a custom terminator flag; # seals it and acts as a recovery anchor. Reserved flags cannot be redefined.

Exercises

14.1 Write a `defk` block that creates the ¤ terminator flag, following the book's pattern.
14.2 Explain why redefining `$` with `defk` is an error.

Chapter 15 — Standard Library

Libraries are loaded using `imp*name.ed` (Sd) or `imp*name.$` (Sd++), and must be placed at the beginning of the file. Using a function from a library that has not been loaded causes error E205.

Library	Function	Main functions (signatures are proposals [PR])
`ioprt`	Screen input/output	`pri`, `ipt` (language keywords)
`math`	Mathematics	`math.abs(x)`, `math.pow(x ' y)`, `math.sqrt(x)`, `math.max(a ' b)`, `math.min(a ' b)`
`mem`	Low-level memory	`mem.memset(p ' value ' cell-count)`, `sta.alloc(bytes)`, `sta.free(p)`
`str`	Strings	`str.len(s)`, `str.upper(s)`, `str.lower(s)`, `str.concat(a ' b)`
`sys`	System	`sys.time()` (seconds), `sys.sleep(ms)`
`file`	Disk files	`file.write(path ' string)`, `file.read(path)`
`draw`	Character-based drawing	`draw.rect(x ' y ' width ' height)`

15.1 Mathematics

Example 15.1. Square root, power, and maximum

```
imp*ioprt.ed
imp*math.ed

sta(main){
    iid: sta(float c) => edt: sta(c = math.sqrt(2.0)) ed
    iid: sta(int p) => edt: sta(p = math.pow(2 ' 10)) ed
    iid: sta(int m) => edt: sta(m = math.max(3 ' 8)) ed
    iid: sta(pri) => "sqrt2=/f3{c} 2^10=/d{p} max=/d{m}" ed
    iid: ref* 0 ed
}
```

Result according to the specification: `sqrt2=1.414 2^10=1024 max=8`.

15.2 Files

Example 15.2. Write a file, then read it back

```
imp*ioprt.ed
imp*file.ed

sta(main){
    iid: call file.write("journal.txt" ' "Sd++ is running") ed
    iid: sta(str s) => edt: sta(s = file.read("journal.txt")) ed
    iid: sta(pri) => "Read back: /s{s}" ed
    iid: ref* 0 ed
}
```

Time and sleep

`sys.time()` returns the number of seconds since a fixed reference point. It can measure execution time by subtracting two calls. `sys.sleep(ms)` pauses the program for `ms` milliseconds.

Chapter summary

Load libraries with `imp*name`. The seven standard libraries are `ioprt`, `math`, `mem`, `str`, `sys`, `file`, and `draw`. Functions are called in the form `library.function(argument ' argument)`.

Exercises

15.1 Read the lengths of two perpendicular sides and print the hypotenuse using `math.sqrt`.
15.2 Write a string to a file, read it back, and print it.

Chapter 16 — The Sd and Sd++ Compiler

This chapter is for readers who want to understand or write a compiler. The original specification targets x86_64 machine code; the practical approach is to write a reference interpreter first, verify the semantics, and only then generate machine code.

16.1 Pipeline

```
Source code
    -> Lexical analysis
    -> Syntax analysis
    -> Semantic checking
    -> Code generation / execution
```

16.2 Lexical analysis

The lexer splits source code into tokens. Sd-specific points include:

- `ed`, `dbl`, `strt`, `void`, `bln`, and `fn` are keywords; `$`, `₫`, `€`, `$$`, `#`, and `##` are terminator flags (match the longest token first: `$$` before `$`, `##` before `#`).
- `=>` is one token; `==` and `=` are two different tokens.
- `'` separates data items; `,` separates specification tags.
- Comments: when the lexer sees an opening `\\`, it skips until the closing `\\`. If the end of file is reached before closure, it reports E101 with the opening line instead of silently consuming the rest of the file.

16.3 Syntax analysis

Sd syntax is very regular: an instruction is `iid: sta(...) [=> tag: ...] ed`; a block has an opening line ending with `:` and a closing line of the form block-name plus `ed`. Therefore, the parser uses a stack of currently open blocks and checks the closing rules as follows:

Pseudocode for checking closing rules

```
stack = []
for each line:
```

```
    if line is a block opening "iid: sta(K) => tag:":
        check that the tag matches
            (for: cdn,chck; while: cdn,edt; ...)
        push closer(K) onto stack       // dbl / strt / void / bln / fn
    if line is "X ed" where X is in {dbl,strt,void,bln,fn}:
        if stack is empty:               error E202
        if top != X:                     error E202 (wrong block closer)
        otherwise:                       pop the top
at end of file:
    if stack still contains elements:   error E202 (unclosed block)
```

16.4 Semantic checking

The most important checks in Sd 2 / Sd++ 2 are:

1. Every variable must be declared before it is used (E301); names must not be duplicated (E302).
2. Types must be valid: a string cannot be assigned to an `int`; conditions in `bolean` and `while` must be `bool` (E303).
3. `main` and every `fn` must end with `ref*` (E204, E207).
4. A `while` loop must modify a variable in its condition (W201); a Reback branch must modify the `sb` variable (W203).
5. Every `sta.alloc` must have a corresponding `sta.free` in the same frame (W302).

16.5 Error recovery

When a syntax error occurs, a good compiler does not stop immediately; it tries to find other errors as well. Sd++ uses **panic-mode recovery**: it skips tokens until a synchronization point, then continues parsing. Synchronization points are: a block-name ed closer, the # flag, or ##. When ## is reached, the compiler cleans up its state and terminates reliably.

16.6 Implementation roadmap

1. Write a lexer and print the token list for the book's examples.
2. Write a parser for Part I, build the syntax tree, and run the block-closing checker.
3. Write a syntax-tree interpreter: variables, operators, `pri`, `ipt`, branching, and loops.
4. Add functions, arrays, and structures; rerun every example and compare the results with those in the book.
5. Add Sd++: pointers, Heap, `sb` with Reback, `define`, and `defk`.
6. Only at the end should you generate machine code or intermediate code.

Chapter summary

Pipeline: lexical analysis, syntax analysis, semantics, code generation. The block-closing rule is checked with a stack. The block-name ed, #, and ## synchronization points help recover after errors. Write a reference interpreter first.

Exercises

16.1 List the tokens in the line `iid: sta(int a) => edt: sta(a = 5) ed`.
16.2 If code is missing two `bln` ed lines, what remains on the compiler's stack at the end of the file?

Chapter 17 — Integrated Projects

17.1 Project 1: Class gradebook (Sd)

This project uses structures, arrays, functions, loops, and branching. The program reads the names and scores of three students, prints a grade table with classifications, and calculates the class average.

Example 17.1. Class gradebook

```
imp*ioprt.ed

iid: sta(struct) => dta:
    sta HocSinh { str ten ' float diem }
strt ed

iid: sta(fn) => rtn:
    sta xeploai(float d) -> str {
        iid: sta(str kq) => edt: sta(kq = "Poor") ed
        iid: sta(bolean) => cdn,tf:
            sta(d >= 8.0) {
                iid: sta(kq = "Excellent") ed
            }
            sta(d >= 6.5) {
                iid: sta(kq = "Good") ed
            }
            sta(d >= 5.0) {
                iid: sta(kq = "Average") ed
            }
        bln ed
        iid: ref* kq ed
    }
fn ed

sta(main){
    iid: sta(HocSinh ds[3]) ed
    iid: sta(float tong) ed
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 0 ' i < 3 ' i++) {
            iid: sta(pri) => "Student name /d{i + 1}:" ed
            iid: sta(ipt) => edt: sta(ds[i].ten) ed
            iid: sta(pri) => "Score:" ed
            iid: sta(ipt) => edt: sta(ds[i].diem) ed
            iid: sta(tong = tong + ds[i].diem) ed
        }
    dbl ed
    iid: sta(pri) => "--- Grade table ---" ed
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 0 ' i < 3 ' i++) {
            iid: sta(str lx) => edt: sta(lx = xeploai(ds[i].diem)) ed
            iid: sta(pri) => "/s{ds[i].ten} /f{ds[i].diem} /s{lx}" ed
        }
    dbl ed
    iid: sta(pri) => "Class average: /f{tong / 3.0}" ed
    iid: ref* 0 ed
}
```

With input `An 7.0`, `Binh 9.0`, and `Chi 4.5`, the output according to the specification is:

```
--- Grade table ---
An 7.00 Good
Binh 9.00 Excellent
Chi 4.50 Poor
Class average: 6.83
```

17.2 Project 2: Self-healing server monitor (Sd++)

This project uses global variables, pointers, the `ids sb` block with Reback, the custom `¥` flag, and the Heap. When a server node's status is negative, the system repeatedly increases it until

it reaches 200 (full operating capacity), then writes a log to the Heap.

Example 17.2. Monitoring and self-healing

```
ssta {
    imp*ioprt.$
    imp*mem.$

    defk<ids.end: text "¥"> {
        init ids.edt ("¥" endlogic == "$"+"€"+"$$")
        func * end.sta * end.iid #
    }

    ids.g: cluster = "Cloud-Cluster-Alpha" ₫
    ids.g: node_status = -100 ₫

    define[
        ids.edt: text = "main",
        pri.edt: text = "print"
    ]{
        main: {
            init ¥
            iptedt.int status = ids(node_status).g ¥
            iptedt.ptr p = @status ¥
            ids.pri: "== MONITOR: /s{ids(cluster).g} ==" ¥
            ids.pri: "Initial status: /d{status}" ¥
            ids sb(status) {
                block status == 200:
                    ids.pri: "[ONLINE] 100% capacity" ¥
                    transf $
                block status < 0:
                    ids.pri: "[WARNING] fault (/d{status}), resetting" ¥
                    *p = *p + 300 ¥
                    $$
                remaining:
                    ids.pri: "[RESET] restoring to 200" ¥
                    *p = 200 ¥
                    $$
            } ¥
            iptedt.ptr log = sta.alloc(16) $
            *log = status $
            ids.pri: "Log written: /d{*log}" $
            sta.free(log) $
            ref*0 ¥:
        } ¥
    } €
}
##
```

Result according to the specification:

```
== MONITOR: Cloud-Cluster-Alpha ==
Initial status: -100
[WARNING] fault (-100), resetting
[ONLINE] 100% capacity
Log written: 200
```

The `status < 0` branch modifies `status` indirectly through pointer `p`. The compiler may issue W203 because it cannot be certain which variable the pointer references; here, however, `p` is assigned `@status`, so the warning is only a reminder.

Exercises

17.1 Extend Project 1: also print the name of the student with the highest score.
17.2 Extend Project 2: if the initial status is 500, restore it to 200 using the `remaining` branch; verify this by tracing each iteration.

Appendix A — Keywords and Symbols

The "Source" column uses **G** for items present in the author's original code and **[PR]** for proposals by the compiler of this textbook, pending approval.

Keyword / symbol	Meaning	Source	Chapter
`sta`	Statement Authority; frame (the body)	G	1
`iid`	Instruction Identity; instruction identifier (the soul)	G	1
`ed`	Instruction / block terminator	G	1
`imp*`	Load a library, in the form `imp*name.ed`	G	1
`main`	Main program frame	G	1
`pri`, `ipt`	Print; input	G	3
`ref*`	Close a frame and return a value	G	1, 7
`=>`	Connect the instruction header to its tag and body	[PR]	1
`cdn`	Condition tag	G	4, 5
`chck`	Step-control tag (`for`)	G	5
`edt`	Value-editing tag	G	2, 5
`dta`	Data tag (`struct`)	G	8
`unkw`	"Outside data" tag (`void`)	G	7
`tf`	True/false tag (`bolean`)	G	4
`rtn`	Return-value tag (`fn`)	[PR]	7
`bolean`	Branching block	G	4
`for`, `while`	Loops	G	5
`struct`	Structure	G	8
`void`	Procedure / procedure-closing marker	G	7
`fn`	Function returning a value / function-closing marker	[PR]	7
`dbl`	Paired family: opens the body and closes a loop	G	5
`strt`	Structure closer	G	8
`bln`	`bolean` closer	G	4
`else`, `brk`, `nxt`, `call`	Fallback branch; exit loop; next iteration; call	[PR]	4, 5, 7

Keyword / symbol	Meaning	Source	Chapter
`and`, `or`, `not`	Logical operators	[PR]	2, 4
`int`, `float`, `str`	Integer, floating-point number, string types	G	2
`bool`, `true`, `false`	Boolean type and values	G / [PR]	2, 4
`/s{}`, `/d{}`, `/f{}`	Formatting placeholders	G (`{ }` is [PR])	3
`/b{}`, `/f3{}`, `//`	Boolean placeholder, three decimal places, slash	[PR]	3
`\\ ... \\`	Block comment	G	1
`*/`	Single-line comment (Sd++)	G	10
`'`	Separates data items	G	5, 6
`,`	Separates specification tags	G	4, 5
`ssta`, `##`	Open and close an Sd++ program	G	10
`$`, `₫`, `€`, `$$`, `#`	Terminator-flag set	G	10
`define[ ]{ } €`	Outer frame and renaming	G	11
`ids.g`, `ids(x).g`	Declare and access a global variable	G	11
`iptedt.int`, `iptedt.ptr`	Flat-form local declaration	G	11
`ptr`, `@x`, `*p`	Pointer, address, dereference	G	12
`sta.alloc`, `sta.free`	Allocate and free Heap memory	G	12
`ids sb`, `block`, `remaining`	Selection block, conditional branch, fallback branch	G	13
`transf`, `break`	Exit a block (`break $` is [PR])	G	13
`defk`, `init`, `func`	Custom terminator flags	G	14

Appendix B — Formal Grammar (EBNF)

Notation: `[ ]` means optional; `{ }` means zero or more repetitions; `|` means choice; text in double quotation marks is literal text. This is a reduced version for Sd 2 (Part I).

Sd 2 grammar

```
program         ::= { import } { unit } main
import          ::= "imp*" NAME ".ed"
unit            ::= function_block | procedure_block | structure_block
main            ::= "sta(main){" { instruction } "iid: ref*" expression "ed" "}"

instruction     ::= simple_instruction | block
simple_instruction ::= "iid:" instruction_content "ed"
instruction_content ::= declaration | assignment | print | input | call | return | jump
```

```
declaration     ::= "sta(" TYPE NAME [ "[" NUMBER "]" ] ")"
                    [ "=>" "edt:" "sta(" NAME "=" value ")" ]
assignment      ::= "sta(" target "=" expression ")"
                    | "sta(" target ("++" | "--") ")"
print           ::= "sta(pri)" "=>" expression
input           ::= "sta(ipt)" "=>" "edt:" "sta(" target ")"
call            ::= "call" NAME "(" [ expression { "'" expression } ] ")"
return          ::= "ref*" expression
jump            ::= "brk" | "nxt"
target          ::= NAME { "[" expression "]" | "." NAME }
value           ::= expression | "[" expression { "'" expression } "]"

block           ::= boolean_block | for_block | while_block
boolean_block   ::= "iid: sta(bolean) => cdn,tf:"
                    { "sta(" expression ") {" { instruction } "}" }
                    [ "sta(else) {" { instruction } "}" ] "bln ed"
for_block       ::= "iid: sta(for) => cdn,chck:"
                    "dbl sta(" [TYPE] NAME "=" expression "'" expression "'" step ") {
                    { instruction } "}" "dbl ed"
while_block     ::= "iid: sta(while) => cdn,edt:"
                    "dbl sta(" expression ") {" { instruction } "}" "dbl ed"
function_block  ::= "iid: sta(fn) => rtn:"
                    "sta" NAME "(" [parameter { "'" parameter }] ")" "->" TYPE "{"
                    { instruction } "iid: ref*" expression "ed" "}" "fn ed"
procedure_block ::= "iid: sta(void) => unkw:"
                    "sta" NAME "(" [parameter { "'" parameter }] ") {"
                    { instruction } "}" "void ed"
structure_block ::= "iid: sta(struct) => dta:"
                    "sta" NAME "{" TYPE NAME { "'" TYPE NAME } "}" "strt ed"
parameter       ::= TYPE NAME [ "[" NUMBER "]" ]
TYPE            ::= "int" | "float" | "str" | "bool" | STRUCT_TYPE
comment         ::= "\\" ... "\\"       (ignored; missing closer -> E101)
```

Specification tags and block closers

Block	Specification tags	Closer	Family
`for`	`cdn,chck`	`dbl ed`	Paired family (two tags)
`while`	`cdn,edt`	`dbl ed`	Paired family
`bolean`	`cdn,tf`	`bln ed`	Logical family
`struct`	`dta`	`strt ed`	Single family (one tag)
`void`	`unkw`	`void ed`	Single family
`fn`	`rtn`	`fn ed`	Function family
`main`	(none)	`ref* ... }`	Closed by a return instruction

Appendix C — Error and Warning Codes

All codes below are proposals [PR] by the compiler of this textbook to make messages consistent; the author may change their numbers.

Code	Meaning	Chapter
Lexical		
E101	A block comment is opened but not closed (reports opening line)	1
E102	Invalid character	16

Code	Meaning	Chapter
E103	String literal is missing its closing quotation mark	16
Syntax		
E201	Missing ed (or terminator flag) at the end of an instruction	9
E202	Block not closed, or closed with the wrong type	4, 16
E203	Missing or extra }	9
E204	`main` does not end with `ref*`	1, 9
E205	A library is used without being imported with `imp*`	15
E206	`for` header is missing initialization, condition, or step	5
E207	Function with a return type is missing its final `ref*`	7
E208	Sd++ program is missing `ssta` or ##	10
E209	A reserved or sealed flag is redefined	14
E210	`defk` block is missing its sealing #	14
Semantic		
E301	Undeclared variable	2
E302	Duplicate name (variable, field, or function)	2, 8, 11
E303	Type mismatch, including use of = as a condition	2
E304	Function called with the wrong number or type of arguments	7
E305	`ref*` used outside a valid frame; `brk` or `nxt` used outside a loop	5, 7
E306	`sta(else)` is not the final branch of a `bolean` block	4
Runtime		
E401	Division by zero	9
E402	Reback exceeds the 65,536-iteration limit	13
E403	Input has the wrong data type	3
E404	Array index is out of range	6
E405	Null pointer dereference	12
E406	Null pointer or already-freed pointer passed to `sta.free`	12
E407	Heap memory exhausted	12
E408	Read or write outside the allocated Heap region	12
Warnings		
W201	`while` loop does not modify a variable in its condition	5

Code	Meaning	Chapter
W202	Old syntax used (`/dtemp`, `break $$`, `transf $ $`, comma-separated array values)	3, 11, 13
W203	Reback branch does not appear to modify the `sb` variable	13
W301	Declared variable or function is never used	2
W302	`sta.alloc` has no corresponding `sta.free`	12

Appendix D — Original Specification Compared with Sd 2

This appendix lists every change from the original code, with reasons. The author may accept individual items or retain the original form for any item they disagree with; the rest of the book is independent except where explicitly noted.

D.1 From screenshots to Sd 2

#	Original form	Problem	Sd 2
1	== connects the header to a tag and is also used for comparison	One symbol has two meanings	=> connects; == is comparison only
2	`sta(int i, ipt) == edt:`	`ipt` is used both as a type and as an input instruction	`ipt` is only an input instruction (Chapter 3)
3	`\\ ... \\` comments; compressed form uses \	Two forms can accidentally consume code	Only `\\ ... \\`; missing closer reports E101 with opening line
4	`sta(i ' i < 10 ' i++)`	Initialization is missing	`sta(int i = 0 ' i < 10 ' i++)`
5	`double sta(int b = 0) == sta(i < 0)`	Declares `b` but tests `i`; wrong from the outset	`dbl sta(condition);` the body must modify the condition variable (W201)
6	`double` opens a loop; `dbl` closes it	Easily confused with a floating-point type	Use `dbl` consistently in both places
7	`arr[int a(10) ' ... float a(1.4)]`	Mixes arrays and structures and duplicates field name a	`sta Name { type name ' ... };` field names must be unique
8	`sta nondta{ int c = 2 ' iid: ... ' float d = 32.2 }`	Mixes declarations and instructions using '; unnamed block	Named procedure: `sta name() { ... }`, called using `call`
9	`sta chcktf (true, false) == sta(a = 0)`	Uses assignment as a condition	`bolean` block with `sta(a == 0)` and `sta(else)`

#	Original form	Problem	Sd 2
10	`sta() diem { ... }`	Repeated name; no parameters or type	`iid: sta(fn) => rtn: ... fn ed`
11	`ref* 0 ed:`	Extra colon	`iid: ref* 0 ed`; mandatory in `main`
12	`imp*ioprt.ed` and `impioprt.ed`	Two spellings	Only `imp*name.ed`
13	`,` and `'` used interchangeably	Their roles are unclear	`,` separates tags; `'` separates data
14	`/dtemp`	Ambiguous name boundary	`/d{temp}`
15	Each block chooses its own closing form	The closing type cannot be determined from the block	Matching closer rule: block-name ed according to family (Appendix B)

D.2 From the old Sd++ specification to Sd++ 2

Original form	Problem	Sd++ 2
`break $$` and `transf $ $`	`$$` means both Reback and “finish”	`$$` means Reback only; exit with `transf $` or `break $`
Reback has no limit	Can loop forever	W203 at compile time; E402 after 65,536 iterations
`sta.arr.g: so = [10, 20] ₫`	Comma is used to separate elements	`[10 ' 20]`; old form warns W202
`/dtemp`, `/sids(x).g`	Ambiguous	`/d{temp}`, `/s{ids(x).g}`
Sd++ and Sd use separate terminator sets	The learner must learn two systems	ed is `$` at instruction level; Sd is a subset of Sd++
`sta.free(p)` says nothing about `p`	Pointer may be reused after freeing	`p` becomes null; E405 and E406
Memory-cell size is unspecified	`alloc(n)` cannot be calculated	One memory cell is 8 bytes
Missing ## has no specific error	Resource cleanup may not occur	E208

Appendix E — Quick Reference

Sd 2: every instruction form on one page

PROGRAM

```
imp*ioprt.ed
sta(main){ ... iid: ref* 0 ed }
```

SIMPLE INSTRUCTIONS (every instruction ends with ed)

```
iid: sta(int a) ed                                \\ declaration \\
iid: sta(int a) => edt: sta(a = 5) ed             \\ declaration + initial value \\
iid: sta(a = a + 1) ed                            \\ assignment (short form); a++ a-- \\
iid: sta(int m[3]) => edt: sta(m = [1 ' 2 ' 3]) ed    \\ array \\
iid: sta(pri) => "a=/d{a} s=/s{t} f=/f{x}" ed       \\ print \\
iid: sta(ipt) => edt: sta(a) ed                     \\ input \\
iid: call name(1 ' 2) ed                             \\ call \\
iid: brk ed     iid: nxt ed                           \\ exit / skip iteration \\
```

BRANCHING

```
iid: sta(bolean) => cdn,tf:
    sta(condition1) { ... }
    sta(condition2) { ... }
    sta(else) { ... }
bln ed
```

LOOPS

```
iid: sta(for) => cdn,chck:
    dbl sta(int i = 0 ' i < n ' i++) { ... }
dbl ed

iid: sta(while) => cdn,edt:
    dbl sta(condition) { ...modify a variable in the condition... }
dbl ed
```

FUNCTION / PROCEDURE / STRUCTURE

```
iid: sta(fn) => rtn:
    sta name(int a ' int b) -> int { ... iid: ref* a + b ed }
fn ed

iid: sta(void) => unkw:
    sta name(str s) { ... }
void ed

iid: sta(struct) => dta:
    sta Name { str a ' int b }
strt ed
```

OPERATORS (high to low): `( ) [ ] .` | `not -` | `* / %` | `+ -` | `< <= > >=` | `== !=` | `and` | `or`
TYPES: `int float str bool` — defaults: `0`, `0.0`, `""`, `false`
COMMENTS: `\\ multi-line \\`
SEPARATORS: `,` between tags; `'` between data items

Sd++: terminator flags and main blocks

```
ssta { ... } ##          protective box: open / close everything
$                         terminate an instruction inside a frame
₫                         terminate a bare global variable
€                         terminate a define frame
#                         seal defk; recovery anchor
$$                        Reback (inside ids sb); outside sb = emergency stop
ids.g: name = value ₫    global variable; access with ids(name).g
iptedt.int a = 5 $       iptedt.ptr p = @a $       *p = 7 $
sta.alloc(n)             sta.free(p)
ids sb(variable) {
    block condition: ... transf $       complete; exit the block
    block condition: ... modify variable $$  Reback: check again from the start
    remaining: ... break $               exit immediately
} $
defk<ids.end: text "¥"> { init ids.edt (...) func * end.sta * end.iid # }
define[ ids.edt: text = "main", pri.edt: text = "print" ]{
    main: { ... } $
} €
```

Appendix F — Exercise Answers

Try the exercises yourself before looking at the answers. Where a problem has several possible solutions, the answer key gives only one. Results are stated according to the specification (see the Document Information section).

Chapter 1

1.1 Answer

```
imp*ioprt.ed

sta(main){
    iid: sta(pri) => "Nguyen Van A" ed
    iid: sta(pri) => "Class 10A1" ed
    iid: ref* 0 ed
}
```

1.2 Add a comment at the beginning of the file, for example:

```
\\ Written: 04/11/2026. Purpose: Chapter 1 exercise. \\
```

1.3 The first line is missing the ed terminator; the second line is missing the colon after `iid`; the third line is correct.

1.4 It is missing `imp*ioprt.ed` (error E205) and `iid: ref* 0 ed` before `}` (error E204).

Chapter 2

2.1 Results: 3, 2, 3.4, 14, 20.

2.2 Answer

```
imp*ioprt.ed

sta(main){
    iid: sta(int a) => edt: sta(a = 12) ed
    iid: sta(int b) => edt: sta(b = 5) ed
    iid: sta(pri) => "sum = /d{a + b}" ed
    iid: sta(pri) => "difference = /d{a - b}" ed
    iid: sta(pri) => "product = /d{a * b}" ed
    iid: sta(pri) => "quotient = /d{a / b}, remainder = /d{a % b}" ed
    iid: ref* 0 ed
}
```

Result: `sum = 17`, `difference = 7`, `product = 60`, `quotient = 2`, `remainder = 2`.

2.3 Answer

```
imp*ioprt.ed

sta(main){
    iid: sta(float r) => edt: sta(r = 2.5) ed
    iid: sta(float pi) => edt: sta(pi = 3.14159) ed
    iid: sta(pri) => "circumference = /f{2.0 * pi * r}" ed
    iid: sta(pri) => "area = /f{pi * r * r}" ed
    iid: ref* 0 ed
}
```

Result: circumference = 15.71, area = 19.63.

2.4 `sta(a = 0)` is an assignment: it changes a to 0 instead of asking whether a equals 0. Sd 2 reports E303 to prevent this confusion.

Chapter 3

3.1 Answer

```
imp*ioprt.ed

sta(main){
    iid: sta(str ten) ed
```

```
    iid: sta(int tuoi) ed
    iid: sta(pri) => "Name:" ed
    iid: sta(ipt) => edt: sta(ten) ed
    iid: sta(pri) => "Age:" ed
    iid: sta(ipt) => edt: sta(tuoi) ed
    iid: sta(pri) => "Hello /s{ten}, next year you will be /d{tuoi + 1}" ed
    iid: ref* 0 ed
}
```

3.2 Answer

```
imp*ioprt.ed

sta(main){
    iid: sta(float dai) ed
    iid: sta(float rong) ed
    iid: sta(ipt) => edt: sta(dai) ed
    iid: sta(ipt) => edt: sta(rong) ed
    iid: sta(pri) => "Perimeter = /f{2.0 * (dai + rong)}" ed
    iid: sta(pri) => "Area = /f{dai * rong}" ed
    iid: ref* 0 ed
}
```

3.3 Answer

```
imp*ioprt.ed

sta(main){
    iid: sta(int n) ed
    iid: sta(ipt) => edt: sta(n) ed
    iid: sta(pri) => "Hundreds: /d{n / 100}" ed
    iid: sta(pri) => "Tens: /d{n / 10 % 10}" ed
    iid: sta(pri) => "Ones: /d{n % 10}" ed
    iid: ref* 0 ed
}
```

Because `/` and `%` have the same precedence, `n / 10 % 10` is evaluated from left to right as `(n / 10) % 10`.

3.4 `iid: sta(pri) => "Ratio: 3//4" ed`

Chapter 4

4.1 Answer

```
imp*ioprt.ed

sta(main){
    iid: sta(int n) ed
    iid: sta(ipt) => edt: sta(n) ed
    iid: sta(bolean) => cdn,tf:
        sta(n % 2 == 0) {
            iid: sta(pri) => "even" ed
        }
        sta(else) {
            iid: sta(pri) => "odd" ed
        }
    bln ed
    iid: ref* 0 ed
}
```

4.2 Answer

```
imp*ioprt.ed

sta(main){
    iid: sta(int a) ed
    iid: sta(int b) ed
    iid: sta(int c) ed
    iid: sta(ipt) => edt: sta(a) ed
    iid: sta(ipt) => edt: sta(b) ed
    iid: sta(ipt) => edt: sta(c) ed
    iid: sta(int m) => edt: sta(m = a) ed
    iid: sta(bolean) => cdn,tf:
        sta(b > m) {
            iid: sta(m = b) ed
        }
    bln ed
    iid: sta(bolean) => cdn,tf:
        sta(c > m) {
            iid: sta(m = c) ed
        }
    bln ed
    iid: sta(pri) => "Largest: /d{m}" ed
    iid: ref* 0 ed
}
```

4.3 Answer (core condition)

```
iid: sta(bolean) => cdn,tf:
    sta(n % 400 == 0 or (n % 4 == 0 and n % 100 != 0)) {
        iid: sta(pri) => "leap year" ed
    }
    sta(else) {
        iid: sta(pri) => "not a leap year" ed
    }
bln ed
```

4.4 Answer

```
imp*ioprt.ed

sta(main){
    iid: sta(float a) ed
    iid: sta(float b) ed
    iid: sta(ipt) => edt: sta(a) ed
    iid: sta(ipt) => edt: sta(b) ed
    iid: sta(bolean) => cdn,tf:
        sta(a != 0.0) {
            iid: sta(pri) => "x = /f{-b / a}" ed
        }
        sta(b == 0.0) {
            iid: sta(pri) => "infinitely many solutions" ed
        }
        sta(else) {
            iid: sta(pri) => "no solution" ed
        }
    bln ed
    iid: ref* 0 ed
}
```

Chapter 5

5.1 Answer

```
sta(main){
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 2 ' i <= 20 ' i = i + 2) {
            iid: sta(pri) => "/d{i}" ed
        }
    dbl ed
    iid: ref* 0 ed
}
```

5.2 Answer

```
imp*ioprt.ed

sta(main){
    iid: sta(int n) ed
    iid: sta(ipt) => edt: sta(n) ed
    iid: sta(int gt) => edt: sta(gt = 1) ed
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 2 ' i <= n ' i++) {
            iid: sta(gt = gt * i) ed
        }
    dbl ed
    iid: sta(pri) => "/d{n}! = /d{gt}" ed
    iid: ref* 0 ed
}
```

5.3 Answer (for `n > 0`)

```
imp*ioprt.ed

sta(main){
    iid: sta(int n) ed
    iid: sta(ipt) => edt: sta(n) ed
    iid: sta(int dem) => edt: sta(dem = 0) ed
    iid: sta(while) => cdn,edt:
        dbl sta(n > 0) {
            iid: sta(dem++) ed
            iid: sta(n = n / 10) ed
        }
    dbl ed
    iid: sta(pri) => "Digit count: /d{dem}" ed
    iid: ref* 0 ed
}
```

5.4 Answer

```
imp*ioprt.ed
imp*str.ed

sta(main){
    iid: sta(str dong) ed
    iid: sta(for) => cdn,chck:
        dbl sta(int k = 1 ' k <= 4 ' k++) {
            iid: sta(dong = str.concat(dong ' "*")) ed
            iid: sta(pri) => "/s{dong}" ed
        }
    dbl ed
    iid: ref* 0 ed
}
```

5.5 The loop body does not make n move toward 0, so `n != 0` remains true forever (W201). Add `iid: sta(n = n - 1) ed` to the body.

Chapter 6

6.1 Answer

```
imp*ioprt.ed

sta(main){
    iid: sta(int a[5]) => edt: sta(a = [3 ' 8 ' 2 ' 6 ' 1]) ed
    iid: sta(int s) => edt: sta(s = 0) ed
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 0 ' i < 5 ' i++) {
            iid: sta(s = s + a[i]) ed
        }
    dbl ed
    iid: sta(pri) => "Sum = /d{s}" ed
    iid: sta(pri) => "Average = /f{s / 5.0}" ed
    iid: ref* 0 ed
}
```

Result: Sum = 20, Average = 4.00.

6.2 Answer

```
imp*ioprt.ed

sta(main){
    iid: sta(int a[5]) => edt: sta(a = [3 ' 8 ' 2 ' 6 ' 1]) ed
    iid: sta(int dem) => edt: sta(dem = 0) ed
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 0 ' i < 5 ' i++) {
            iid: sta(bolean) => cdn,tf:
                sta(a[i] % 2 == 0) {
                    iid: sta(dem++) ed
                }
            bln ed
        }
    dbl ed
    iid: sta(pri) => "Number of even elements: /d{dem}" ed
    iid: ref* 0 ed
}
```

Result: Number of even elements: 3.

6.3 Answer (core loop)

```
iid: sta(for) => cdn,chck:
    dbl sta(int i = 0 ' i < 2 ' i++) {
        iid: sta(int t) => edt: sta(t = a[i]) ed
        iid: sta(a[i] = a[4 - i]) ed
        iid: sta(a[4 - i] = t) ed
    }
dbl ed
```

6.4 See Answer 5.4.

6.5 Answer

```
imp*ioprt.ed
imp*str.ed

sta(main){
    iid: sta(str s) ed
    iid: sta(ipt) => edt: sta(s) ed
    iid: sta(bolean) => cdn,tf:
        sta(str.len(s) == 0) {
```

```
            iid: sta(pri) => "empty" ed
        }
        sta(else) {
            iid: sta(pri) => "Length: /d{str.len(s)}" ed
        }
    bln ed
    iid: ref* 0 ed
}
```

Chapter 7

7.1 Answer

```
imp*ioprt.ed

iid: sta(fn) => rtn:
    sta lonnhat(int a ' int b) -> int {
        iid: sta(int m) => edt: sta(m = a) ed
        iid: sta(bolean) => cdn,tf:
            sta(b > a) {
                iid: sta(m = b) ed
            }
        bln ed
        iid: ref* m ed
    }
fn ed

sta(main){
    iid: sta(pri) => "Largest: /d{lonnhat(lonnhat(4 ' 9) ' 6)}" ed
    iid: ref* 0 ed
}
```

7.2 Answer

```
imp*ioprt.ed

iid: sta(fn) => rtn:
    sta lachinhphuong(int n) -> bool {
        iid: sta(bool kq) => edt: sta(kq = false) ed
        iid: sta(for) => cdn,chck:
            dbl sta(int k = 1 ' k * k <= n ' k++) {
                iid: sta(bolean) => cdn,tf:
                    sta(k * k == n) {
                        iid: sta(kq = true) ed
                    }
                bln ed
            }
        dbl ed
        iid: ref* kq ed
    }
fn ed

sta(main){
    iid: sta(for) => cdn,chck:
        dbl sta(int i = 1 ' i <= 50 ' i++) {
            iid: sta(bolean) => cdn,tf:
                sta(lachinhphuong(i)) {
                    iid: sta(pri) => "/d{i}" ed
                }
            bln ed
        }
```

```
    dbl ed
    iid: ref* 0 ed
}
```

Result: 1, 4, 9, 16, 25, 36, 49, each on its own line.

7.3 Answer

```
imp*ioprt.ed
imp*str.ed

iid: sta(void) => unkw:
    sta invach(int n) {
        iid: sta(str s) ed
        iid: sta(for) => cdn,chck:
            dbl sta(int i = 0 ' i < n ' i++) {
                iid: sta(s = str.concat(s ' "-")) ed
            }
        dbl ed
        iid: sta(pri) => "/s{s}" ed
    }
void ed

sta(main){
    iid: call invach(10) ed
    iid: ref* 0 ed
}
```

7.4 Answer

```
imp*ioprt.ed

iid: sta(fn) => rtn:
    sta fib(int n) -> int {
        iid: sta(bolean) => cdn,tf:
            sta(n <= 1) {
                iid: ref* n ed
            }
        bln ed
        iid: ref* fib(n - 1) + fib(n - 2) ed
    }
fn ed

sta(main){
    iid: sta(pri) => "fib(10) = /d{fib(10)}" ed
    iid: ref* 0 ed
}
```

Result: `fib(10) = 55`.

Chapter 8

8.1 Answer

```
imp*ioprt.ed

iid: sta(struct) => dta:
    sta PhanSo { int tu ' int mau }
strt ed

iid: sta(fn) => rtn:
    sta cong(PhanSo a ' PhanSo b) -> PhanSo {
        iid: sta(PhanSo r) ed
```

```
        iid: sta(r.tu = a.tu * b.mau + b.tu * a.mau) ed
        iid: sta(r.mau = a.mau * b.mau) ed
        iid: ref* r ed
    }
fn ed

sta(main){
    iid: sta(PhanSo x) ed
    iid: sta(PhanSo y) ed
    iid: sta(x.tu = 1) ed
    iid: sta(x.mau = 2) ed
    iid: sta(y.tu = 1) ed
    iid: sta(y.mau = 3) ed
    iid: sta(PhanSo z) => edt: sta(z = cong(x ' y)) ed
    iid: sta(pri) => "/d{z.tu} over /d{z.mau}" ed
    iid: ref* 0 ed
}
```

Result: `5 over 6`.

8.2 See Project 1 in Chapter 17; omit the classification part and print the class average.

8.3 The field hang appears twice: error E302 (field names must be unique).

8.4 `SinhVien` is a type name, not a variable. You must declare a variable with `sta(SinhVien sv)` and then assign `sv.ten`.

Chapter 9

9.1 E201 (missing terminator).

9.2 Answer

```
imp*ioprt.ed

sta(main){
    iid: sta(int a) ed
    iid: sta(int b) ed
    iid: sta(ipt) => edt: sta(a) ed
    iid: sta(ipt) => edt: sta(b) ed
    iid: sta(bolean) => cdn,tf:
        sta(b == 0) {
            iid: sta(pri) => "division by zero" ed
            iid: ref* 3 ed
        }
    bln ed
    iid: sta(pri) => "Quotient = /d{a / b}" ed
    iid: ref* 0 ed
}
```

9.3 Error E404: a five-element array has indices from 0 through 4.

9.4 Answer

```
imp*ioprt.ed

sta(main){
    iid: sta(int n) => edt: sta(n = 10) ed
    iid: sta(while) => cdn,edt:
        dbl sta(n != 0) {
            iid: sta(pri) => "/d{n}" ed
            iid: sta(n = n - 1) ed
        }
    dbl ed
```

```
    iid: ref* 0 ed
}
```

Chapters 10–12

10.1 Answer 10.1 (the `main` frame)

```
main: {
    ids.pri: "Line one" $
    ids.pri: "Line two" $
    ref*0 $:
} $
```

10.2 (a) ₫ (b) $ (c) €.

10.3 Error E208; without the closing lid, cleanup of the Stack, registers, and Heap is not guaranteed.

11.1 Answer

```
ids.g: ten = "An" ₫
ids.g: tuoi = 20 ₫
ids.pri: "/s{ids(ten).g}, /d{ids(tuoi).g} years old" $
```

11.2 A global variable is outside every frame, so there is no enclosing scope to terminate; it uses its own flag, ₫. A variable inside a frame ends with $, like every other instruction.

11.3 Answer

```
iid: sta(int a) => edt: sta(a = 5) ed
iid: sta(pri) => "hi" ed
```

12.1 Answer (the `main` frame)

```
main: {
    iptedt.int x = 3 $
    iptedt.int y = 8 $
    iptedt.ptr px = @x $
    iptedt.ptr py = @y $
    iptedt.int t = *px $
    *px = *py $
    *py = t $
    ids.pri: "x=/d{x} y=/d{y}" $
    ref*0 $:
} $
```

Result: `x=8 y=3`.

12.2 Answer (the `main` frame)

```
main: {
    iptedt.ptr h = sta.alloc(32) $
    *h = 10 $
    *(h + 1) = 20 $
    *(h + 2) = 30 $
    *(h + 3) = 40 $
    ids.pri: "Sum = /d{*h + *(h + 1) + *(h + 2) + *(h + 3)}" $
    sta.free(h) $
    ref*0 $:
} $
```

Result: `Sum = 100`.

12.3 After `sta.free(h)`, h becomes null; `*h = 1` causes E405.

Chapters 13–17

13.1 Answer (the `main` frame)

```
main: {
    iptedt.int score = -15 $
    ids sb(score) {
        block score > 0:
            ids.pri: "Score review successful: /d{score}" $
            transf $
        block score <= 0:
            ids.pri: "Score /d{score}, add 30" $
            score = score + 30 $
            $$
    } $
    ref*0 $:
} $
```

Result: prints `Score -15, add 30`, then `Score review successful: 15`.

13.2 Answer

```
imp*ioprt.ed

sta(main){
    iid: sta(int score) => edt: sta(score = -15) ed
    iid: sta(while) => cdn,edt:
        dbl sta(score <= 0) {
            iid: sta(pri) => "Score /d{score}, add 30" ed
            iid: sta(score = score + 30) ed
        }
    dbl ed
    iid: sta(pri) => "Score review successful: /d{score}" ed
    iid: ref* 0 ed
}
```

13.3 If the variable does not change, the condition does not change, so Reback repeats forever. The compiler warns with W203; if the program still runs, it stops at E402.

14.1 Answer

```
defk<ids.end: text "¤"> {
    init ids.edt ("¤" endlogic == "$"+"€"+"$$")
    func * end.sta * end.iid #
}
```

14.2 `$` is a reserved flag; redefining it changes the meaning of every program (error E209).

15.1 Answer

```
imp*ioprt.ed
imp*math.ed

sta(main){
    iid: sta(float a) ed
    iid: sta(float b) ed
    iid: sta(ipt) => edt: sta(a) ed
    iid: sta(ipt) => edt: sta(b) ed
    iid: sta(float c) => edt: sta(c = math.sqrt(a * a + b * b)) ed
    iid: sta(pri) => "Hypotenuse = /f{c}" ed
    iid: ref* 0 ed
}
```

15.2 Use the file-write/read example in Chapter 15, changing the string content.

16.1 `iid : sta ( int a ) => edt : sta ( a = 5 ) ed`.

16.2 Two `bln` entries remain on the stack; report E202 (unclosed block) for each.

17.1 Answer (add before `ref*` in `main`)

```
iid: sta(int vt) => edt: sta(vt = 0) ed
iid: sta(for) => cdn,chck:
    dbl sta(int i = 1 ' i < 3 ' i++) {
        iid: sta(bolean) => cdn,tf:
            sta(ds[i].diem > ds[vt].diem) {
                iid: sta(vt = i) ed
            }
        bln ed
    }
dbl ed
iid: sta(pri) => "Highest: /s{ds[vt].ten}" ed
```

17.2 Iteration 1: no `block` branch is true (500 is not 200 and is not negative), so `remaining` runs, sets the value to 200, and performs Reback. Iteration 2: `status == 200`; print `ONLINE` and use `transf` to exit the block.

Appendix G — Glossary

Term	Meaning
Variable	A named memory cell with a data type.
Terminator (ed)	A marker that ends an instruction or block.
Terminator flag	A symbol with a specific termination role in Sd++: `$`, `₫`, `€`, `$$`, `#`, `##`.
Execution specification (tag)	A word such as `cdn`, `chck`, `edt`, `dta`, `unkw`, `tf`, or `rtn` that indicates what a block does.
Error code	A return value indicating that a program did not terminate normally (`ref* N`).
Address	The location of a memory cell; obtained with `@x`.
Heap	A memory region requested at runtime using `sta.alloc`.
Block family	A group of blocks with the same closing method: paired (`dbl`), single (`strt`, `void`), logical (`bln`), and function (`fn`).
Matching closure	The rule that every opened block must be closed with the correct family-specific block-name ed.
Lexer, parser	Lexical analyzer; syntax analyzer.
Pointer	A variable that contains the address of another memory cell.
Reback	The action of returning to the beginning of an `ids sb` block to recheck conditions, marked with `$$`.
Token	The smallest unit of source code after lexical analysis.
[PR]	Marker for a proposal by the textbook compiler, awaiting the author's approval.